AMA 564 Deep Learning

2026 Spring

Lecture 9

Overview

- **Recurrent Neural Networks**
- **Long Short-Term Memory**
- **Word Embedding**

Recurrent Neural Networks

Translation

About 4,180,000,000 results (0.29 seconds)

English ⇄ Chinese (Traditional)

he is eating a hot dog

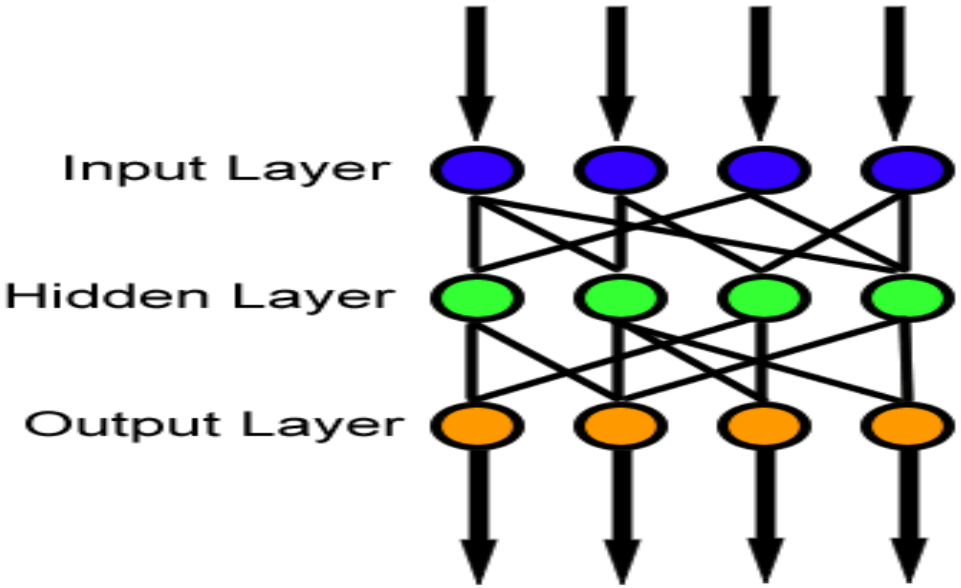
他在吃熱狗

Tā zài chī règǒu

Open in Google Translate • Feedback

Translation

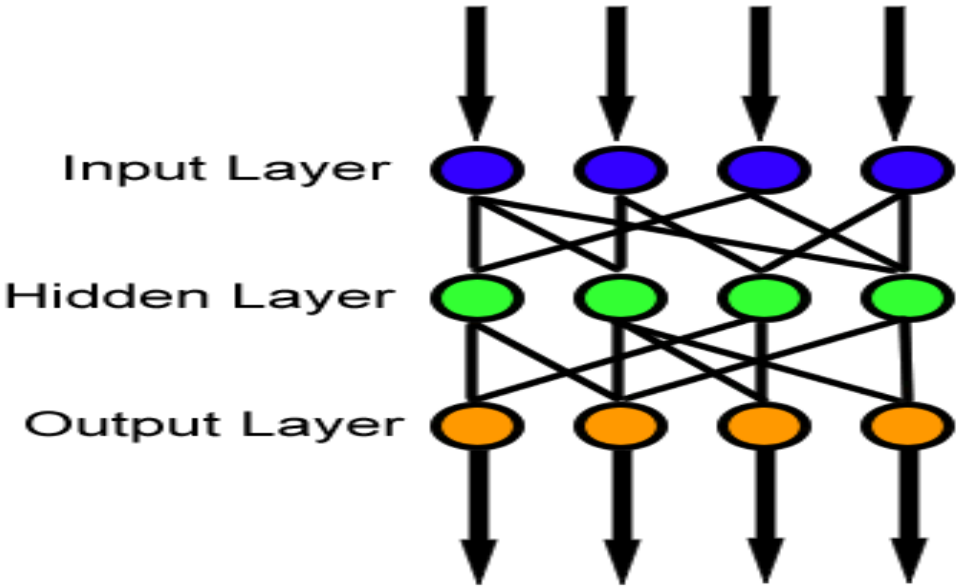
He is eating a hot dog



他 在 吃 热 狗

Translation

He is a lucky dog



他 是 个 幸运 儿

Motivating examples

He is eating a hot dog 6 words

He is a lucky dog 5 words

Two sentences have different length

Motivation:

the neural network should work on data with different length

Motivating examples

He is eating a hot dog

He is a lucky dog

The word ‘dog’ has different meanings.

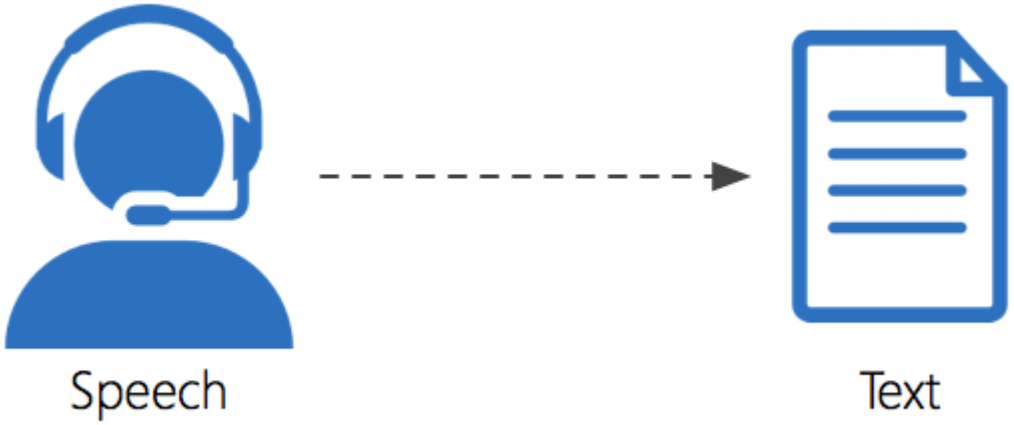
The translation depends on the words before ‘dog’

hot dog **lucky dog**

Motivation:

the neural network should work with sequential-dependent data

Speech to Text



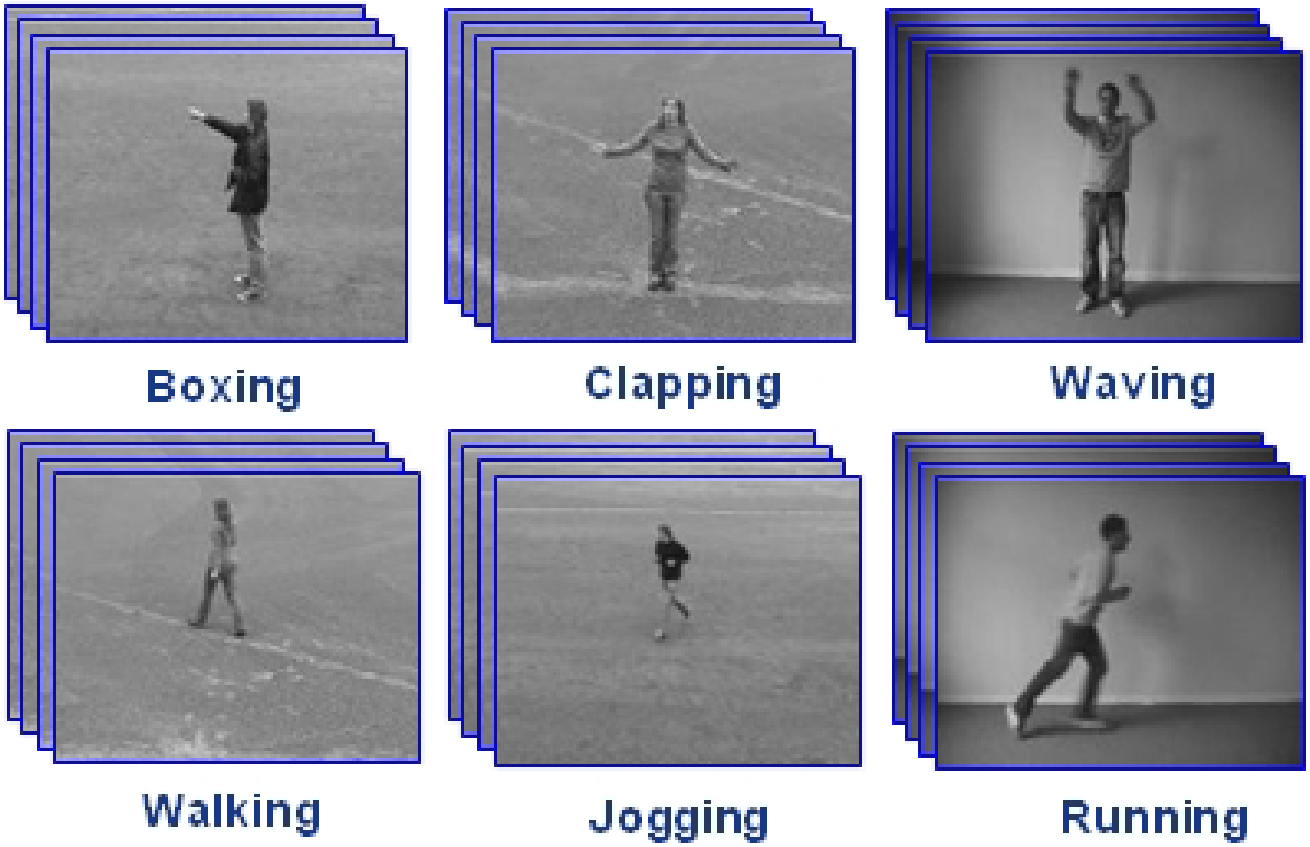
Sentiment classification

"I love this movie.
I've seen it many times
and it's still awesome."

"This movie is bad.
I don't like it it all.
It's terrible."

Motivating examples

Video activity classification



Recurrent Neural Network

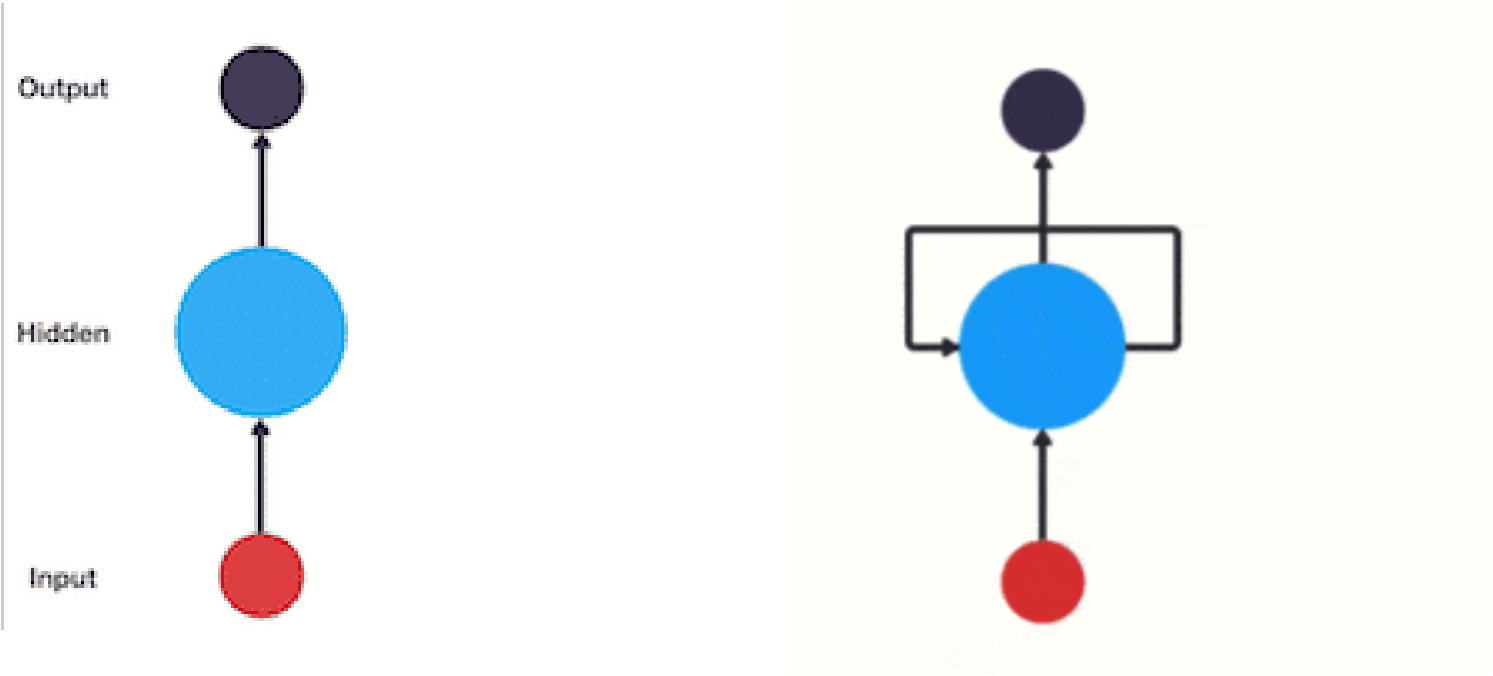
- **Recurrent Neural Networks (RNNs)** are the modern standard to deal with **time-dependent** and/or **sequence-dependent** problems.

- This type of network is “recurrent” in the sense that they can **revisit or reuse past states as inputs to predict the next or future states**.

- They have **memory.** Memory is what allows us to incorporate our past thoughts and behaviors into our future thoughts and behaviors.

Recurrent Neural Network

- The structure of traditional neural networks is relatively simple: input layer-hidden layer-output layer.
- The biggest difference between RNN and traditional neural networks is that each time the previous output is brought to the next hidden layer and trained together.



Source:https://www.easyai.tech/en/ai-definition/rnn/

An example

If we need to judge the user's intention from his/her speak (asking the weather, asking the time, setting an alarm...),

The user says, “**What time is it**?"

We segment this sentence firstly.

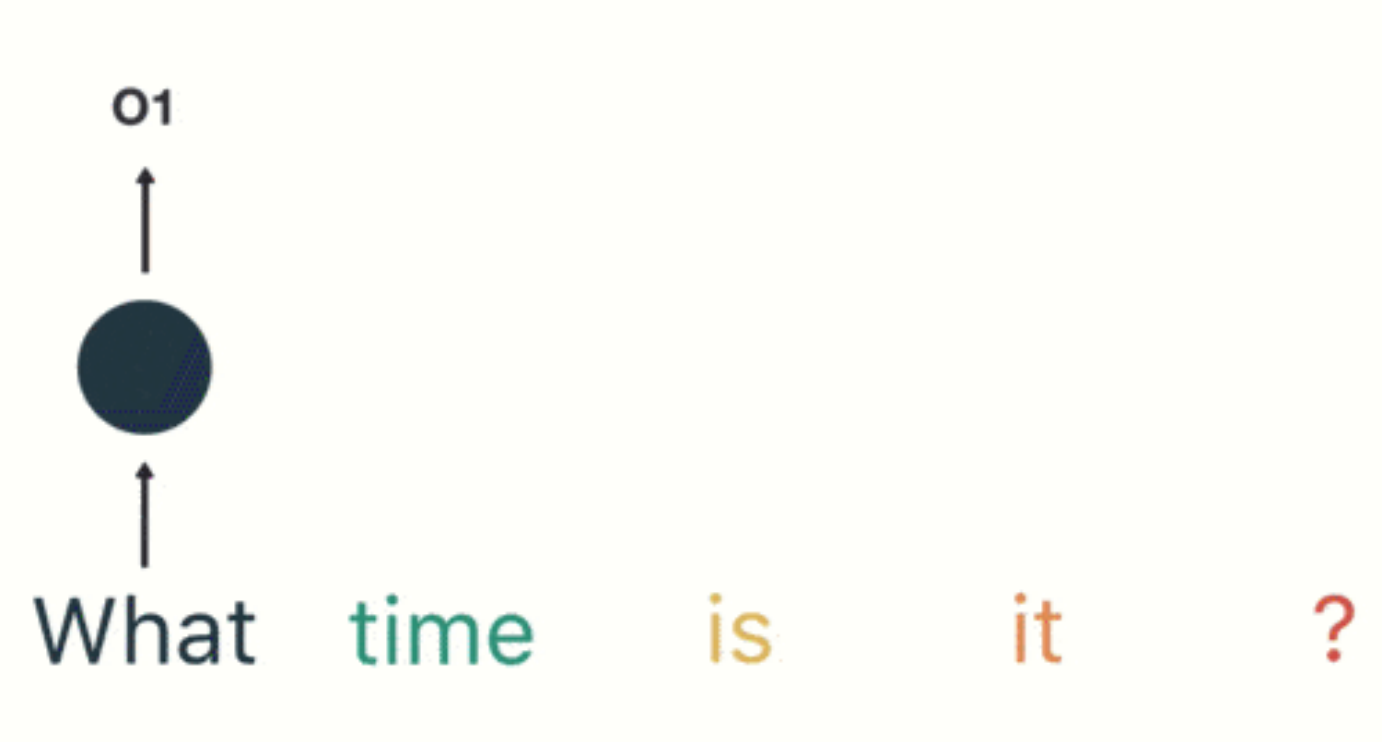
An examples

Input the words into the RNN in order.

First use "**what**" as the input of the RNN and get the output "$\boldsymbol{o_1}$"

An examples

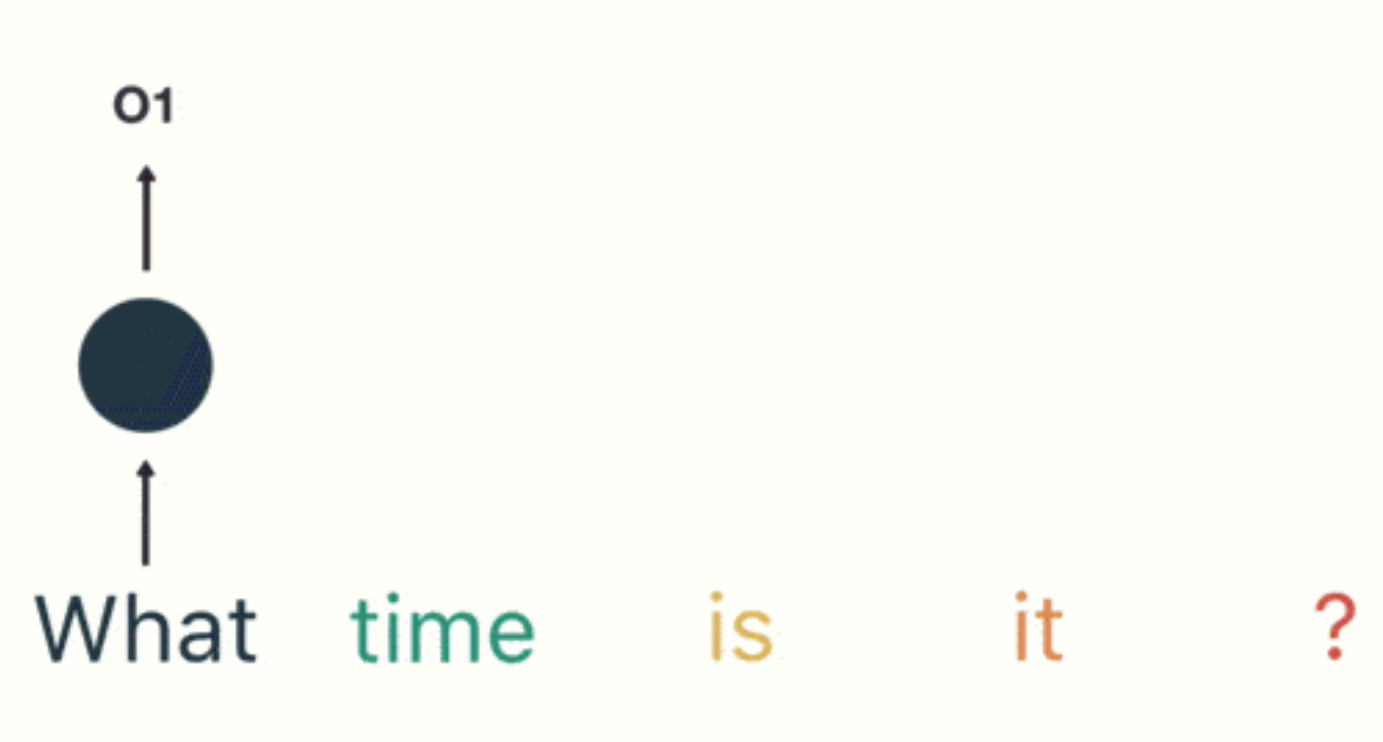
Then input "**time**" to the RNN network in order, and get output "o_2"



An examples

Then input "**time**" to the RNN network in order, and get output "o_2“

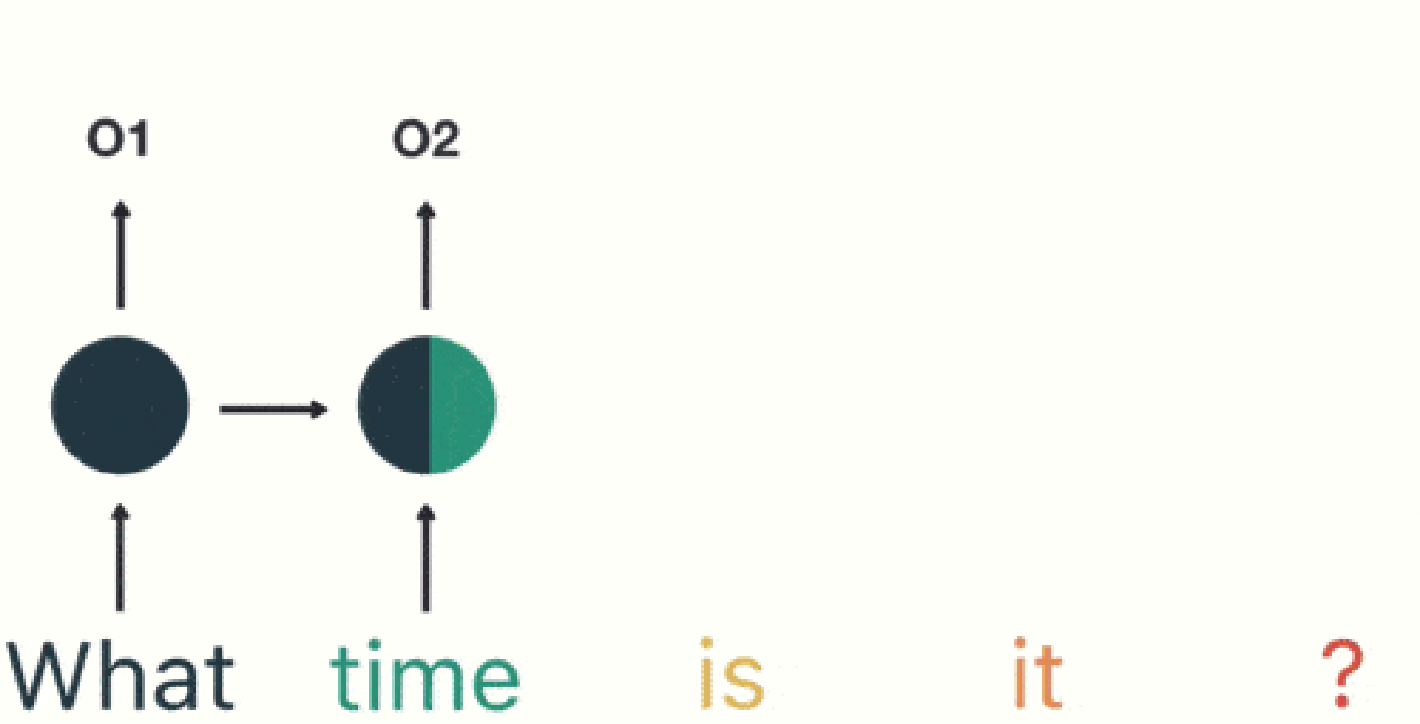
When "**time**" enters, the previous hidden layer of "**what**" also has an effect (part of the hidden layer "**time**" of was black).



An examples

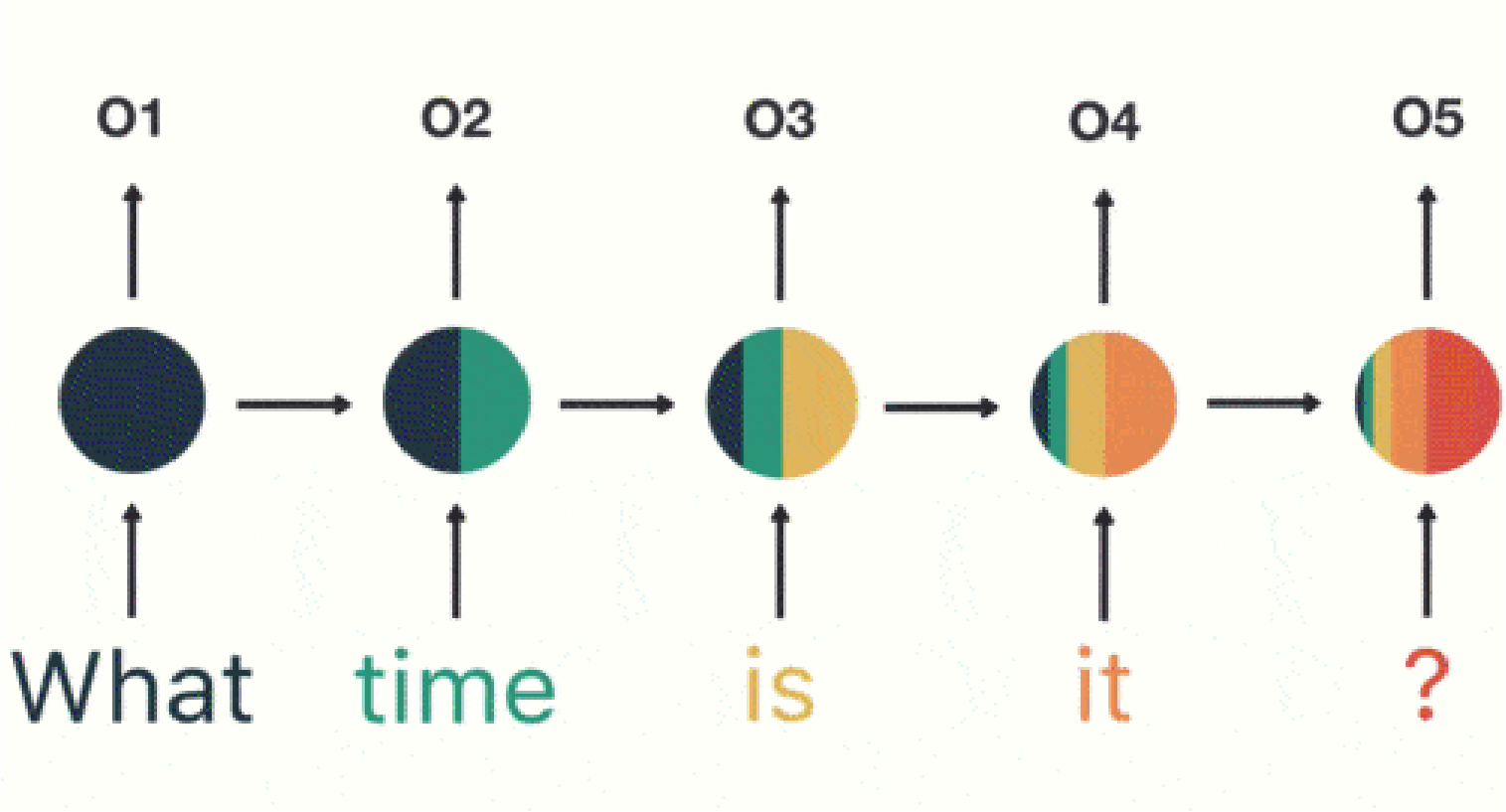
By analogy, all the previous inputs have an impact on the future output.

We can see that the circular hidden layer contains all the previous colors. As shown below:

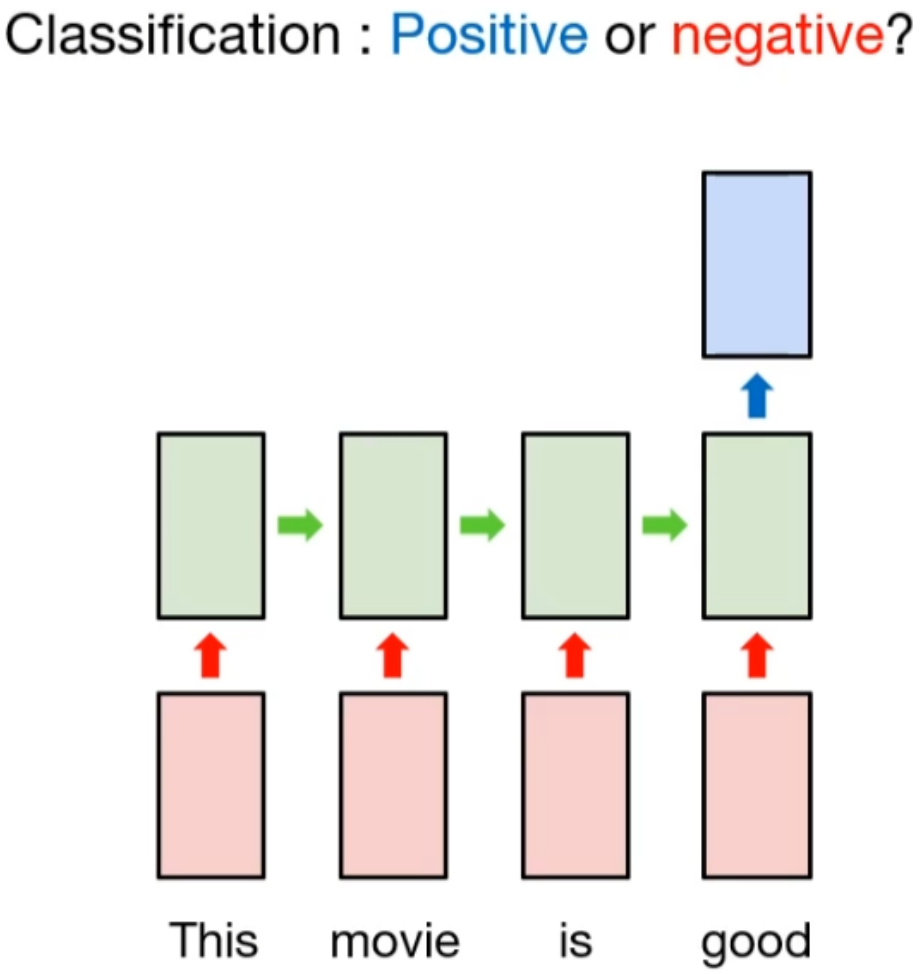


An examples

Lastly, to judge the user's intention, we only need the output " o_5 " in the last layer:

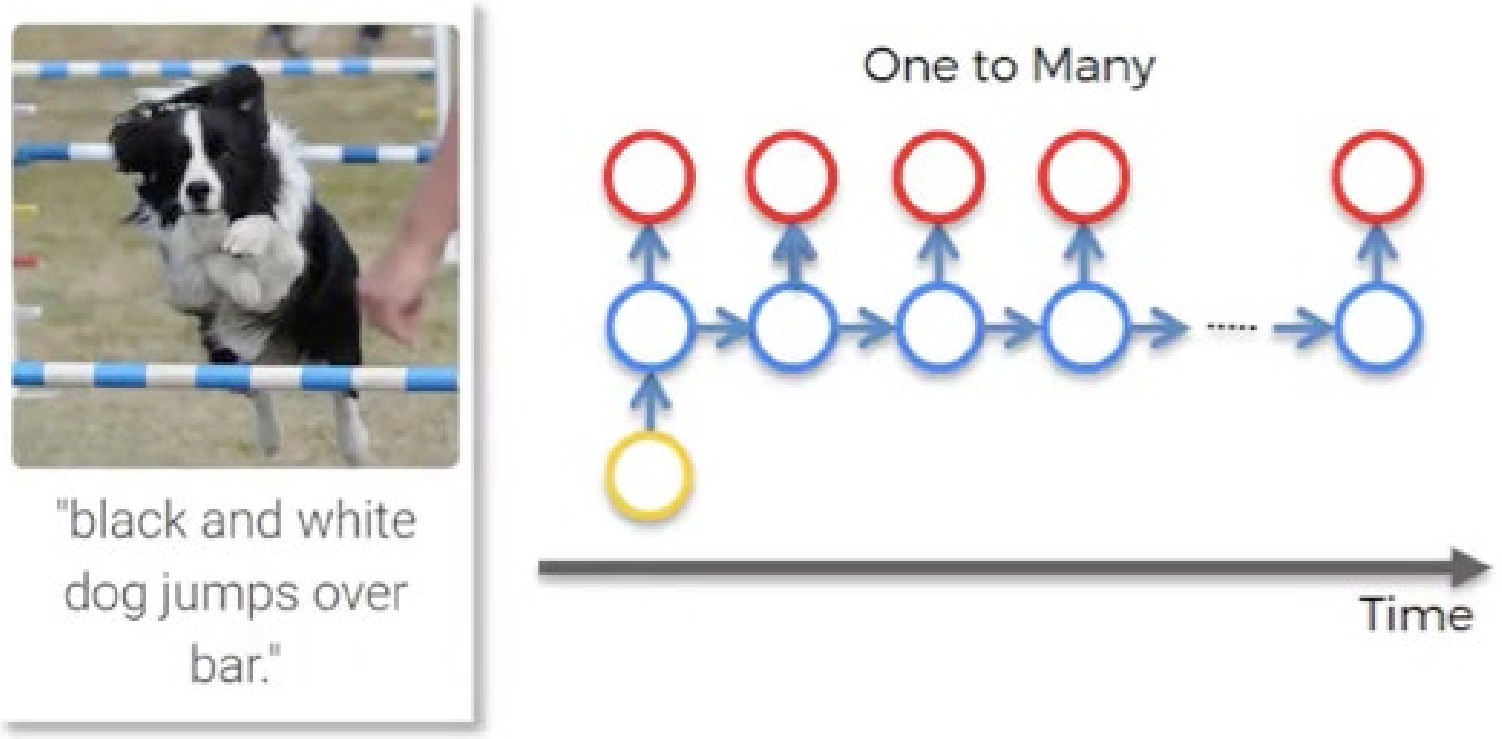


Many-to-one RNN



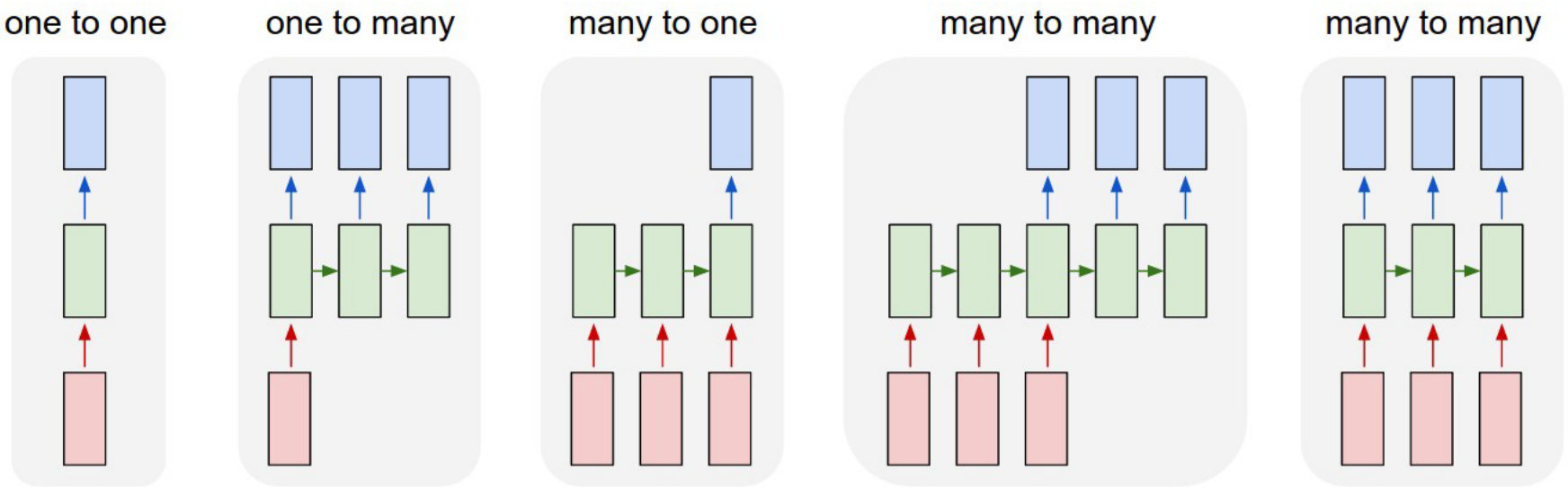
Source: *https://goodboychan.github.io/python/deep_learning/tensorflow-keras/2020/12/06/01-RNN-Many-to-one.html*

One-to-Many RNN



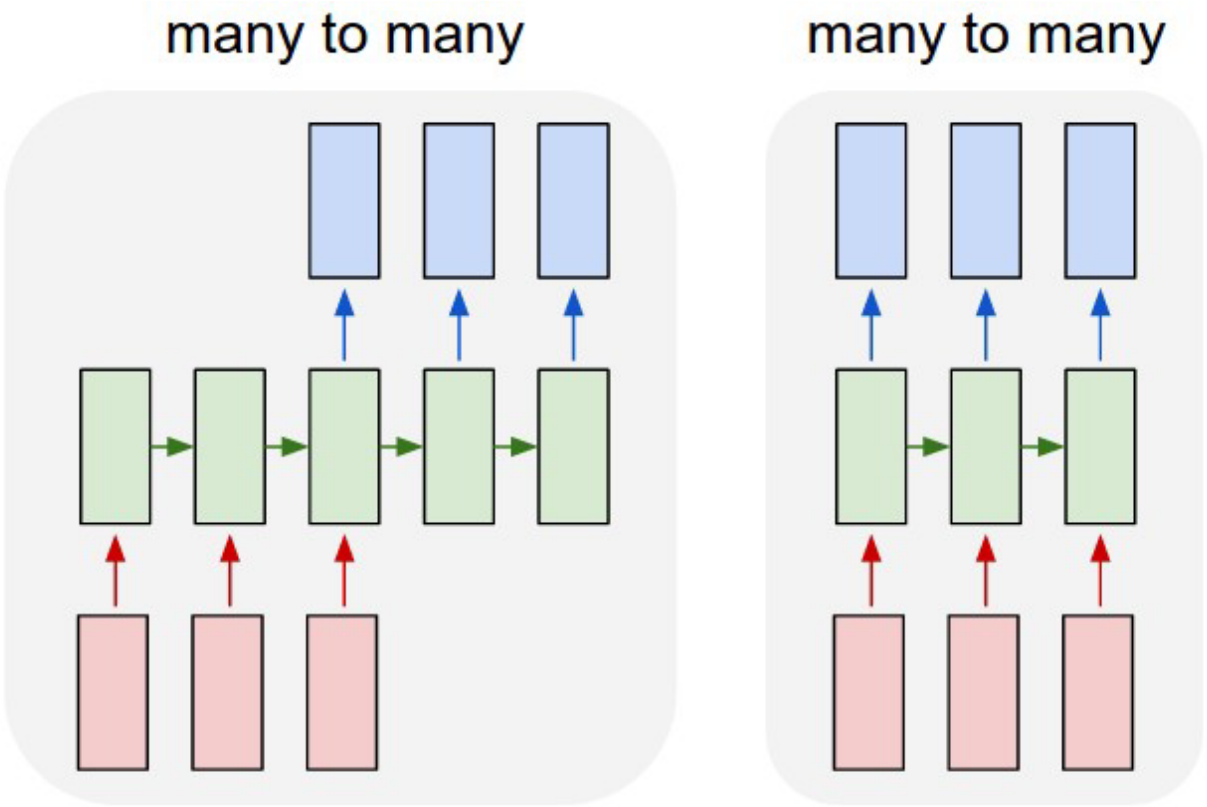
Source: *https://www.superdatascience.com/blogs/recurrent-neural-networks-rnn-the-idea-behind-recurrent-neural-networks*

Different Types of RNN



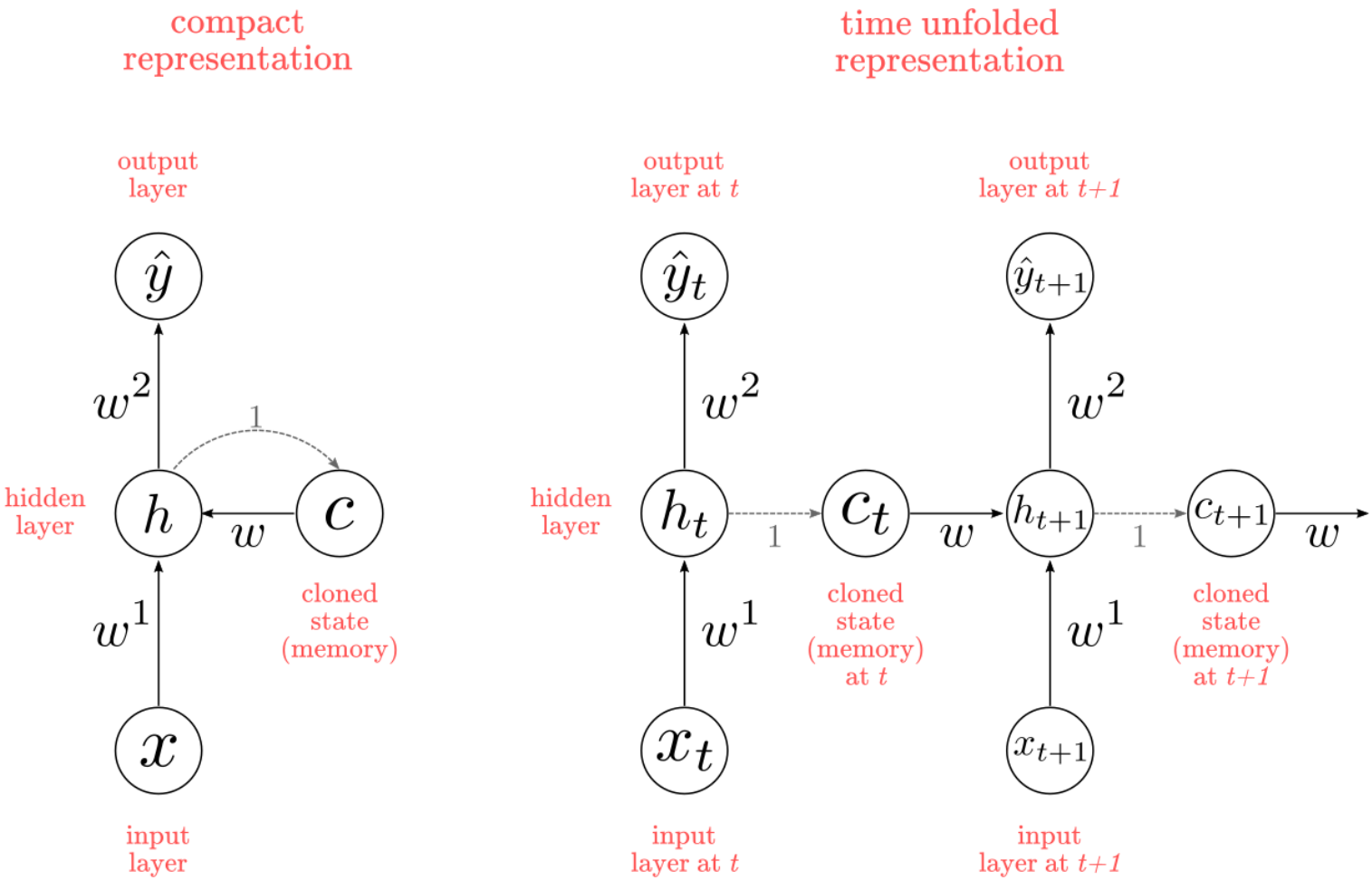
Source: http://karpathy.github.io/2015/05/21/rnn-effectiveness/

Many-to-Many RNN



Source: http://karpathy.github.io/2015/05/21/rnn-effectiveness/

Elman Network



Elman Network

X_1: He → $h_1 = \sigma_h(W_h X_1 + U_h h_0 + b_h)$ → $Y_1 = \sigma_y(W_y h_1 + b_y)$

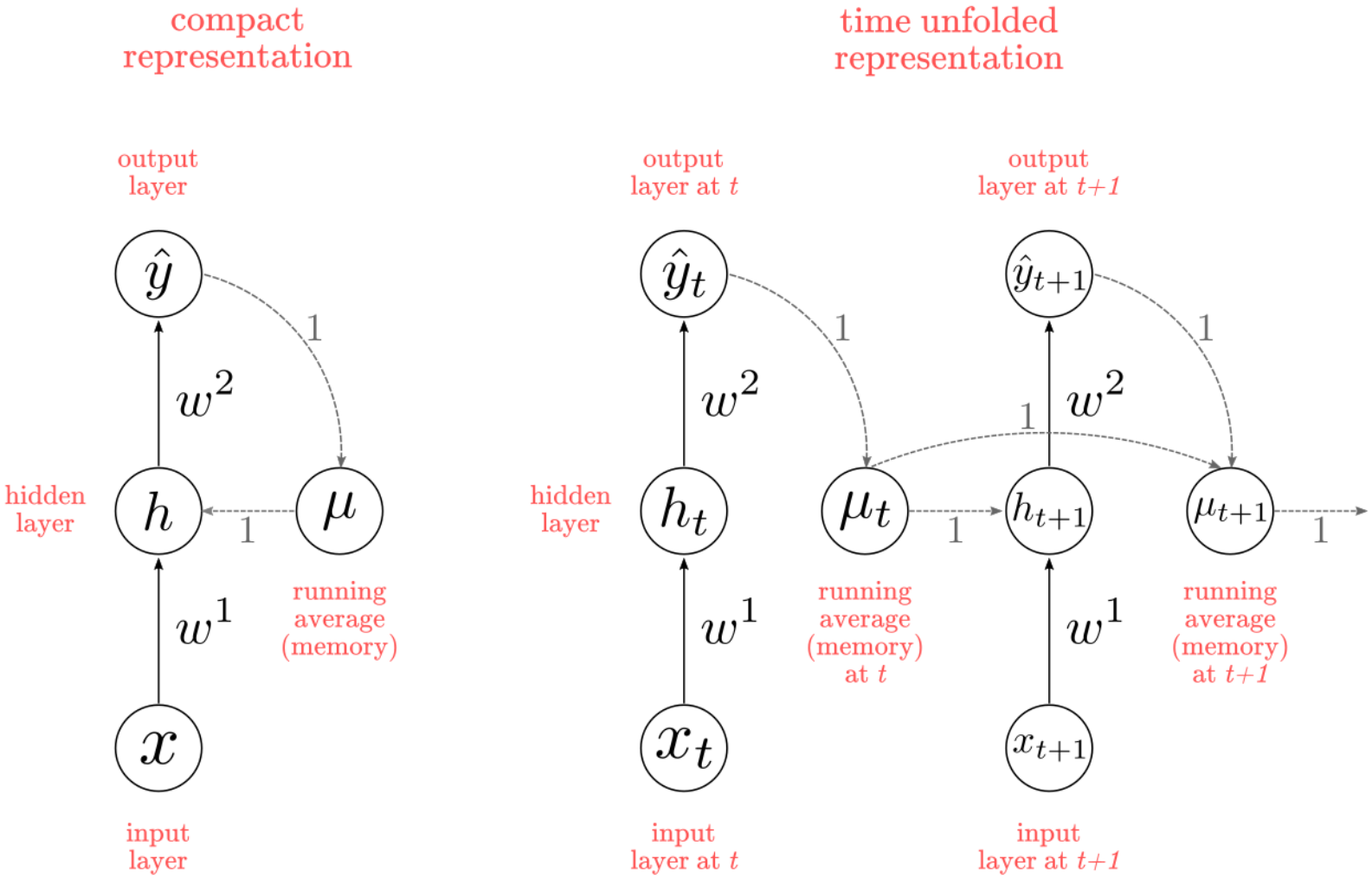
X_2: is → $h_2 = \sigma_h(W_h X_2 + U_h h_1 + b_h)$ → $Y_2 = \sigma_y(W_y h_2 + b_y)$

X_3: a → $h_3 = \sigma_h(W_h X_3 + U_h h_2 + b_h)$ → $Y_3 = \sigma_y(W_y h_3 + b_y)$

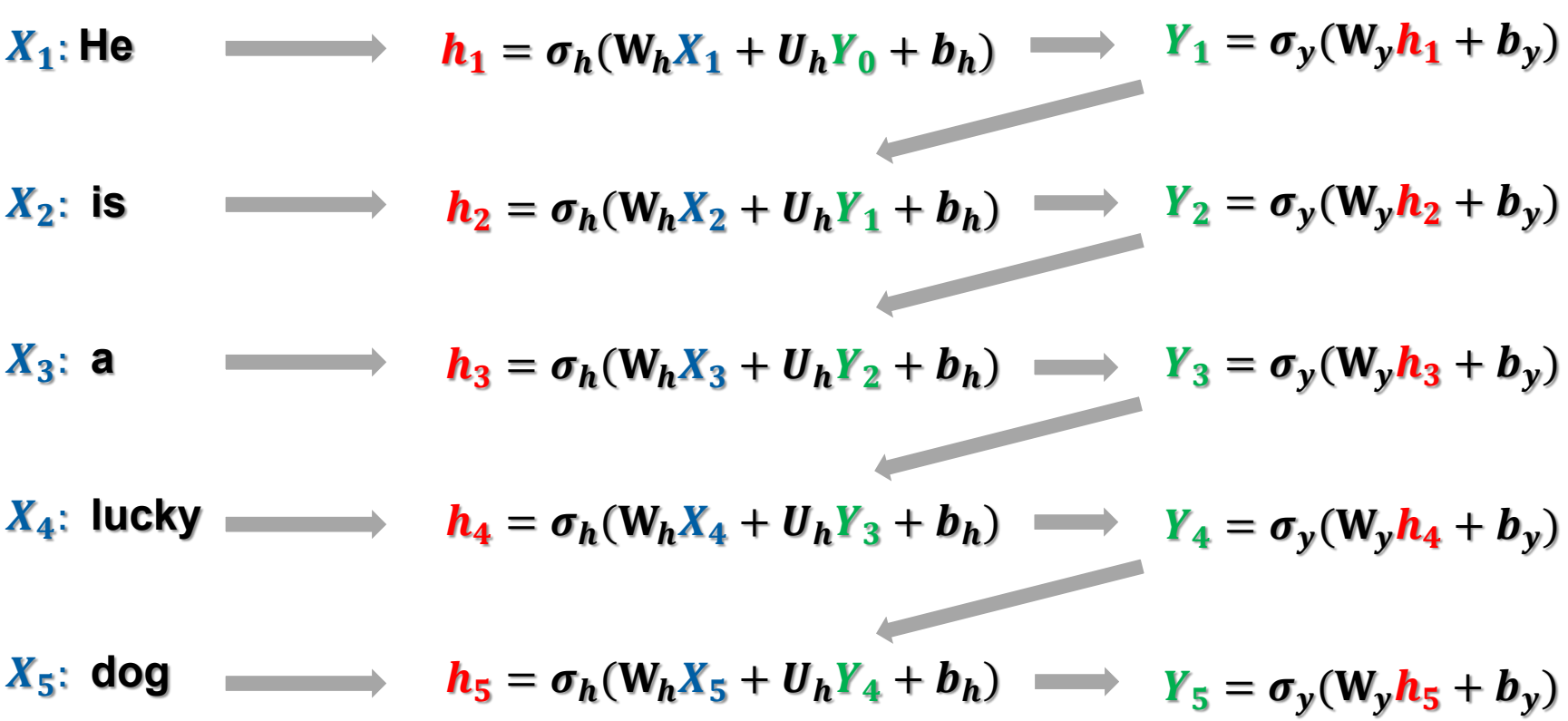
X_4: lucky → $h_4 = \sigma_h(W_h X_4 + U_h h_3 + b_h)$ → $Y_4 = \sigma_y(W_y h_4 + b_y)$

X_5: dog → $h_5 = \sigma_h(W_h X_5 + U_h h_4 + b_h)$ → $Y_5 = \sigma_y(W_y h_5 + b_y)$

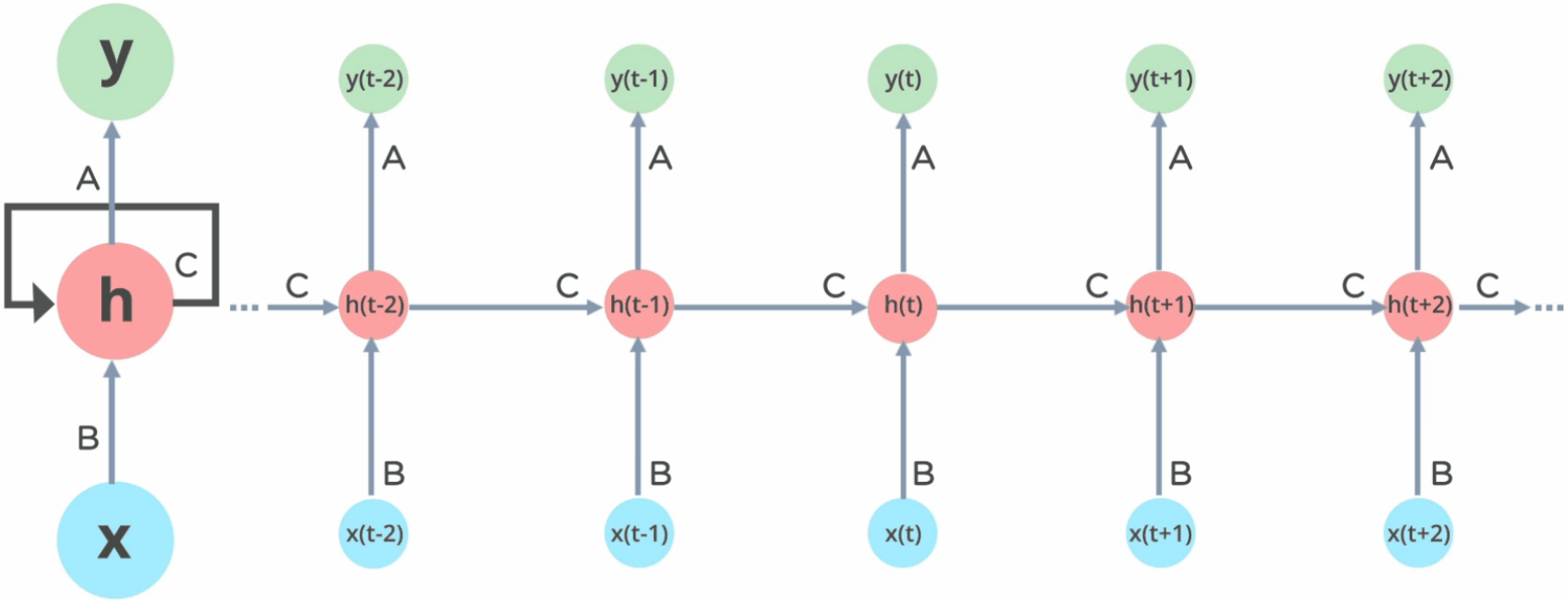
Jordan Network



Jordan Network



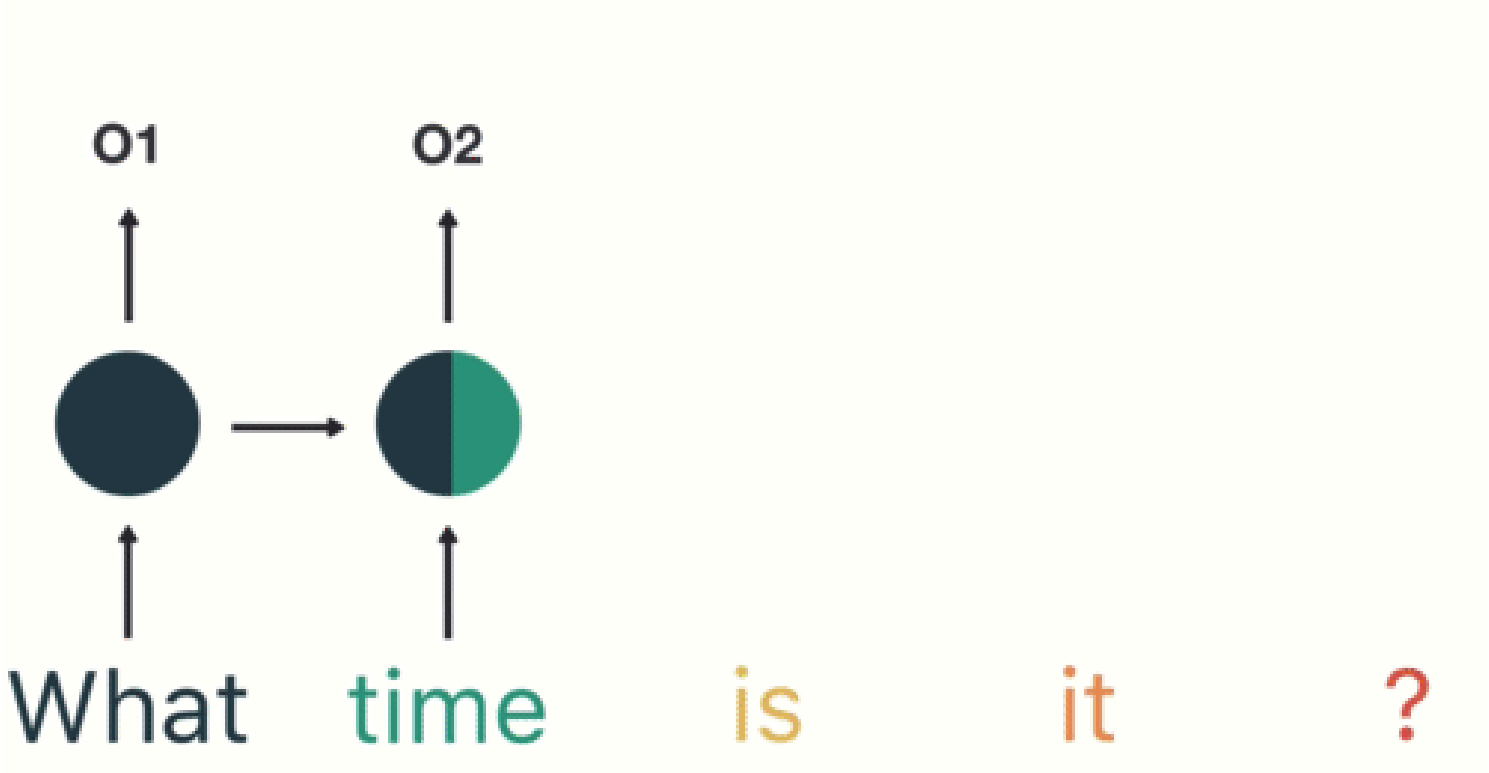
RNN Unfolded over time



A dynamical system

Source: *Simplilearn.com*

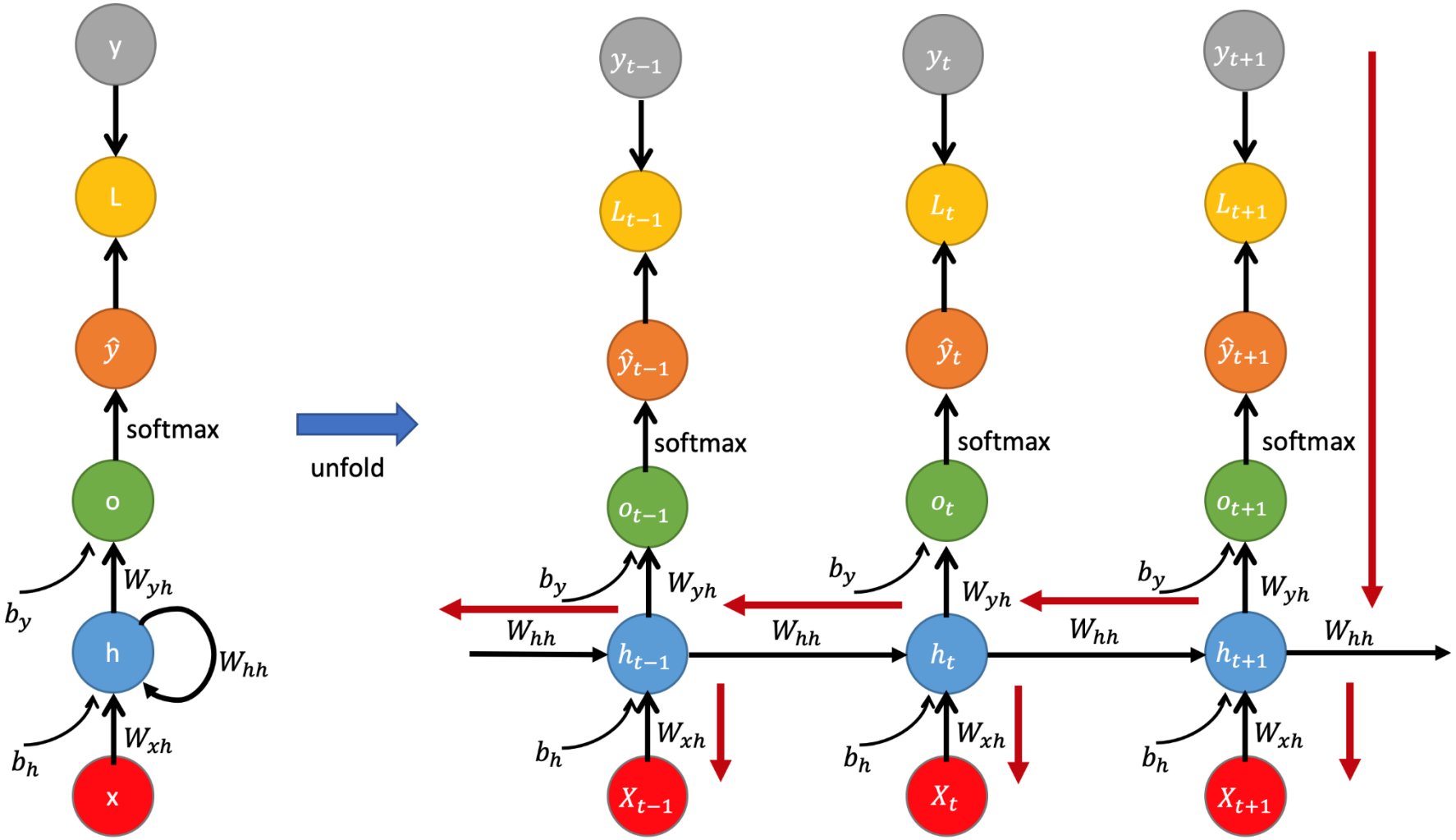
RNN Unfolded over time



A dynamical system

Source: *https://towardsdatascience.com/illustrated-guide-to-recurrent-neural-networks-79e5eb8049c9*

Backpropagation through time



Source: *https://mmuratarat.github.io/2019-02-07/bptt-of-rnn*

Backpropagation through time

The RNN is defined by

$$h_t = f_h(X_t, h_{t-1}) = \phi_h(W_{xh}^T \cdot X_t + W_{hh}^T \cdot h_{t-1} + b_h)$$
$$\hat{y}_t = f_o(h_t) = \phi_o(W_{yh}^T \cdot h_t + b_y)$$

where $\boldsymbol{W_{xh}}$, $\boldsymbol{W_{hh}}$ and $\boldsymbol{W_{yh}}$ are weight matrices for the input, recurrent connections, and the output, respectively and $\boldsymbol{\phi_h}$ and $\boldsymbol{\phi_o}$ are element-wise nonlinear functions.

The Loss function is defined by

$$\begin{aligned} L(\hat{y}, y) &= \sum_{t=1}^{T} L_t(\hat{y}_t, y_t) \\ &= -\sum_{t}^{T} y_t \log \hat{y}_t \\ &= -\sum_{t=1}^{T} y_t log\,[softmax(o_t)] \end{aligned}$$

Backpropagation through time

Note that the weight $\boldsymbol{W_{yh}}$ is shared across all the time sequence. Therefore, we can differentiate to it at the each time step and sum all together:

$$\begin{aligned}\frac{\partial L}{\partial W_{yh}} &= \sum_t^T \frac{\partial L_t}{\partial W_{yh}} \\ &= \sum_t^T \frac{\partial L_t}{\partial \hat{y}_t} \frac{\partial \hat{y}_t}{\partial o_t} \frac{\partial o_t}{\partial W_{yh}}\end{aligned}$$

Similarly, we can get the gradient with respect to bias $\boldsymbol{b_y}$:

$$\begin{aligned}\frac{\partial L}{\partial b_y} &= \sum_t^T \frac{\partial L_t}{\partial \hat{y}_t} \frac{\partial \hat{y}_t}{\partial o_t} \frac{\partial o_t}{\partial b_y} \\ &= \sum_t^T (\hat{y}_t - y_t)\end{aligned}$$

Backpropagation through time

Now, let's go through the details to derive the gradient with respect to $\boldsymbol{W_{hh}}$, considering at the time step $t+1$

$$\frac{\partial L_{t+1}}{\partial W_{hh}} = \frac{\partial L_{t+1}}{\partial \hat{y}_{t+1}} \frac{\partial \hat{y}_{t+1}}{\partial h_{t+1}} \frac{\partial h_{t+1}}{\partial W_{hh}}$$

But, the hidden state $\boldsymbol{h_{t+1}}$ partially depends also on $\boldsymbol{h_t, h_{t-1}, \ldots, h_1}$

$$\frac{\partial L_{t+1}}{\partial W_{hh}} = \sum_{k=1}^{t+1} \frac{\partial L_{t+1}}{\partial \hat{y}_{t+1}} \frac{\partial \hat{y}_{t+1}}{\partial h_{t+1}} \frac{\partial h_{t+1}}{\partial h_k} \frac{\partial h_k}{\partial W_{hh}}$$

Aggregate the gradients with respect to $\boldsymbol{W_{hh}}$ over the whole time-steps with backpropagation,

$$\frac{\partial L}{\partial W_{hh}} = \sum_{t}^{T} \sum_{k=1}^{t+1} \frac{\partial L_{t+1}}{\partial \hat{y}_{t+1}} \frac{\partial \hat{y}_{t+1}}{\partial h_{t+1}} \frac{\partial h_{t+1}}{\partial h_k} \frac{\partial h_k}{\partial W_{hh}}$$

Backpropagation through time

Further, we can take the derivative with respect to $\boldsymbol{W_{xh}}$ over the whole sequence as

$$\frac{\partial L}{\partial W_{xh}} = \sum_{t}^{T} \sum_{k=1}^{t+1} \frac{\partial L_{t+1}}{\partial \hat{y}_{t+1}} \frac{\partial \hat{y}_{t+1}}{\partial h_{t+1}} \frac{\partial h_{t+1}}{\partial h_k} \frac{\partial h_k}{\partial W_{xh}}$$

Vanishing or exploding gradient

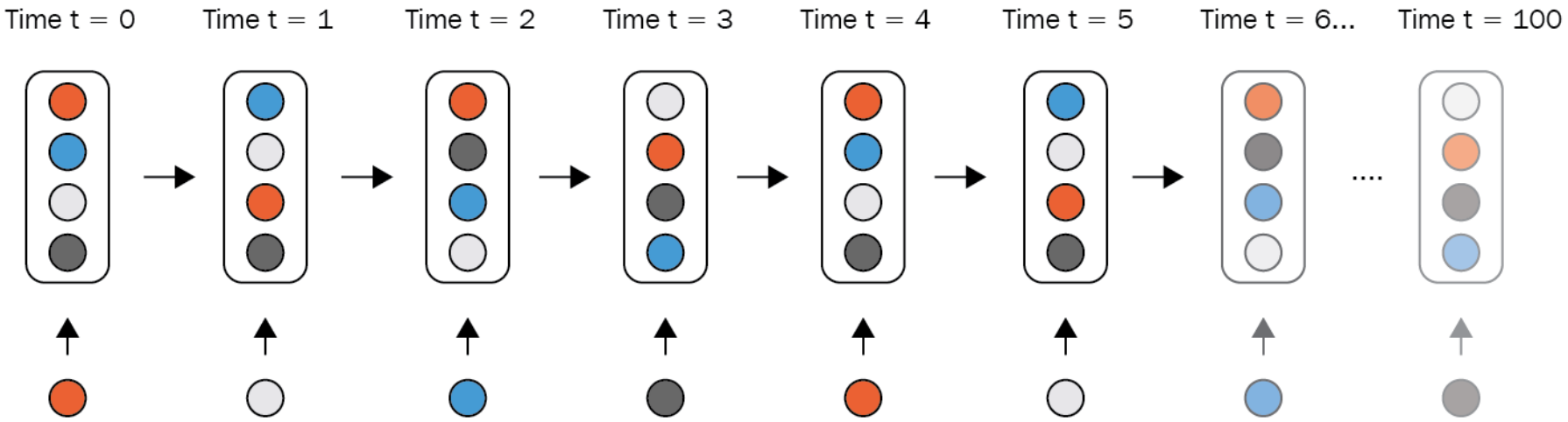
Note: Because of recursive derivative, we need to compute

$$\prod_{j=k}^{t} \frac{\partial h_{j+1}}{\partial h_j} = \frac{\partial h_{t+1}}{\partial h_k} = \frac{\partial h_{t+1}}{\partial h_t} \frac{\partial h_t}{\partial h_{t-1}} \cdots \frac{\partial h_{k+1}}{\partial h_k}$$

This can lead to vanishing gradient or exploding gradient.

Vanishing or exploding gradient

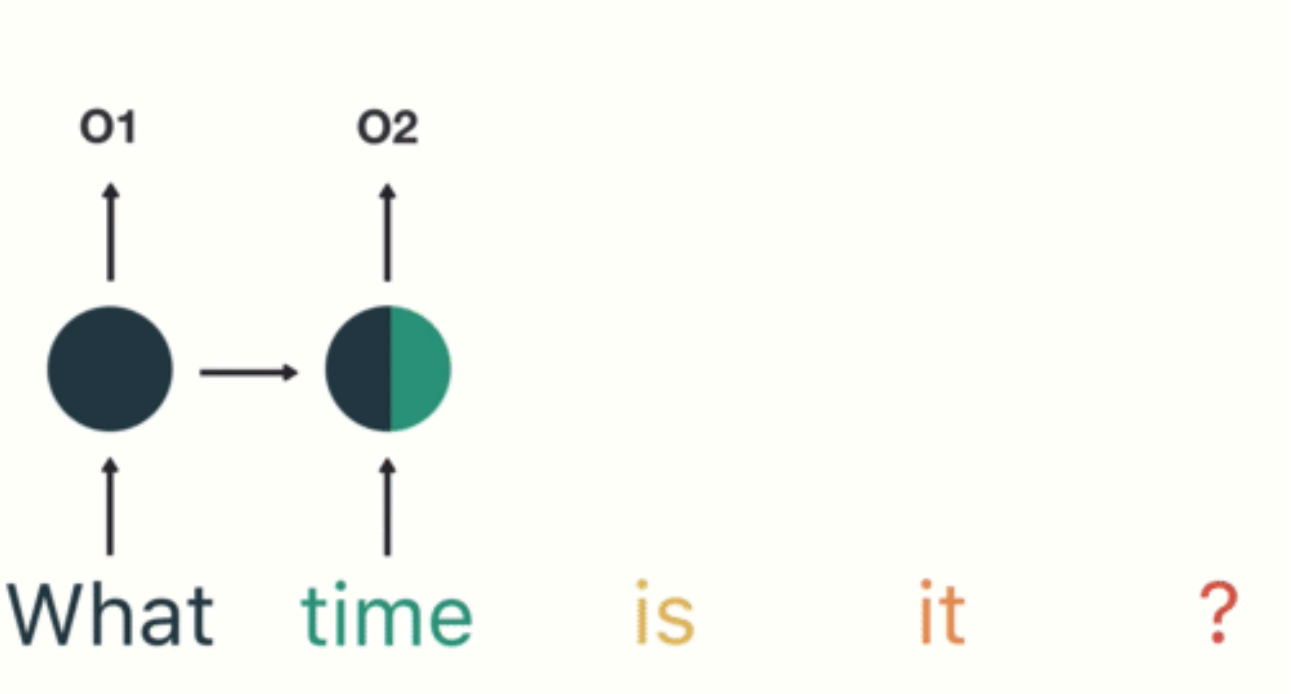
Decay of information through time



Source: O'Reilly Media

Vanishing or exploding gradient

Decay of information through time



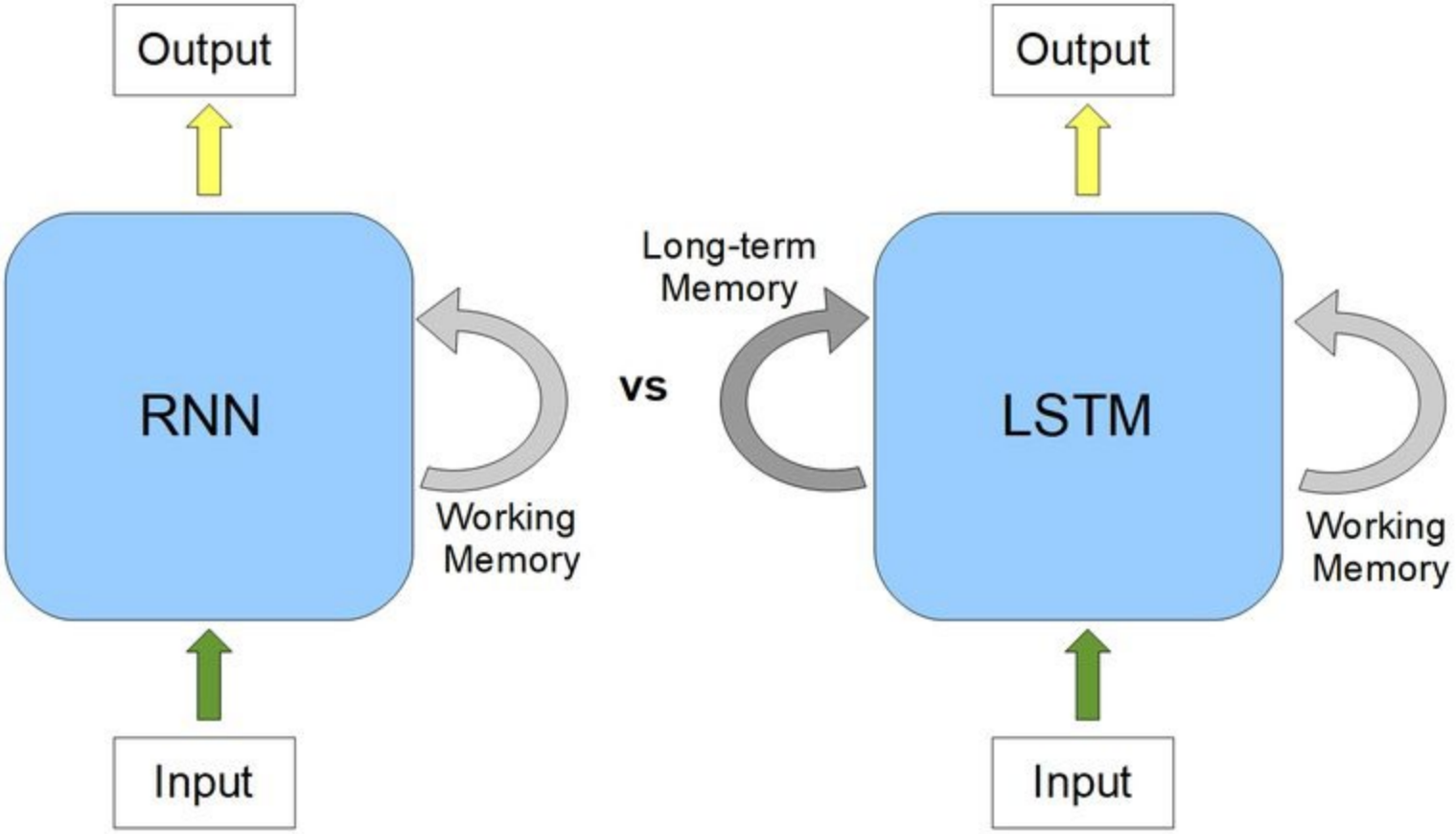
Long Short-Term Memory

Long Short-Term Memory

- The intuition behind the LSTM architecture is to create an additional module in a neural network that learns when to remember and when to forget pertinent information.

- In other words, the network, effectively learns which information might be needed later on in a sequence and when that information is no longer needed.

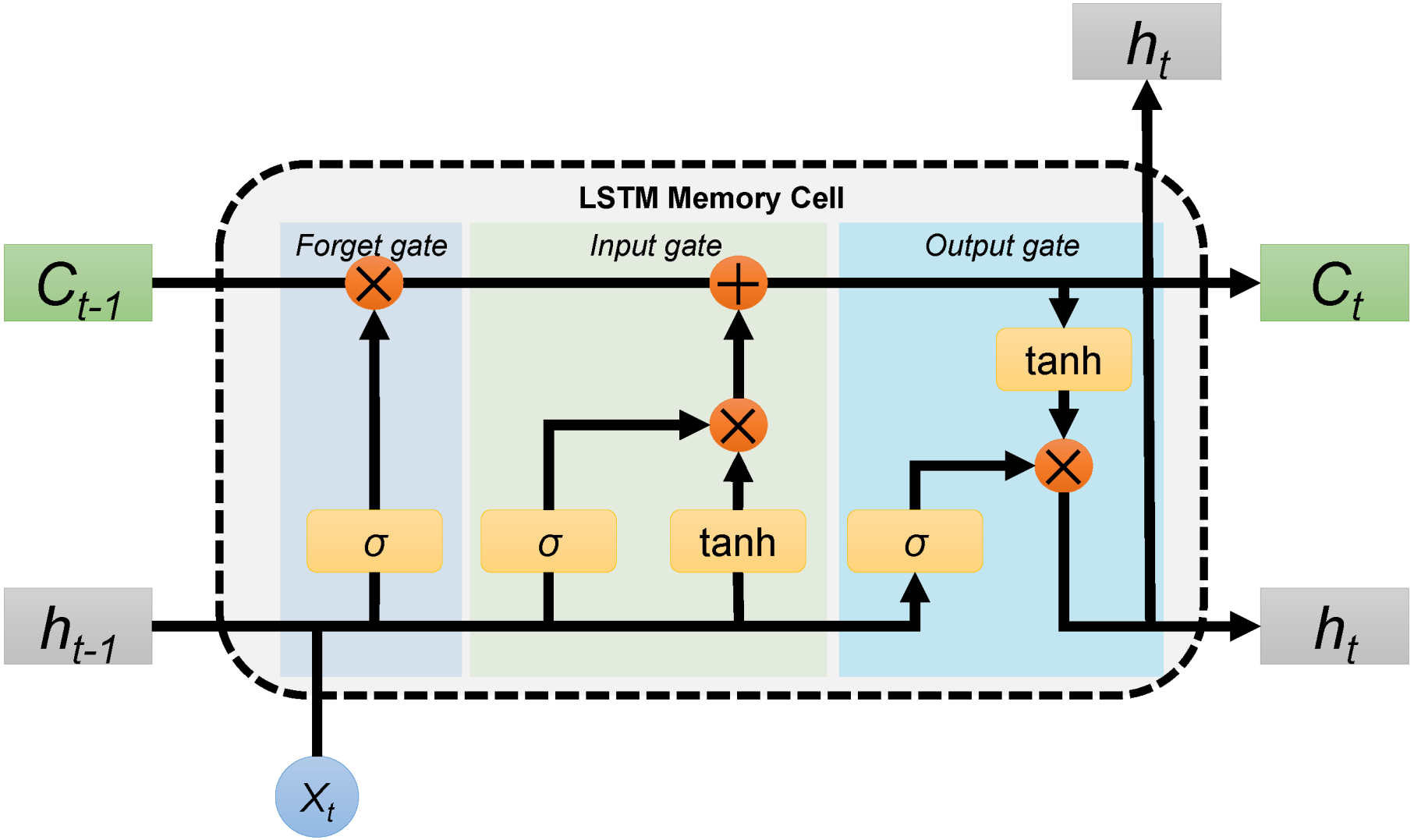
Long Short-Term Memory



Source: Research Gate

Long Short-Term Memory

In addition to the hidden state $\boldsymbol{h_t}$, LSTM also keeps a memory cell $\boldsymbol{c_t}$



Long Short-Term Memory

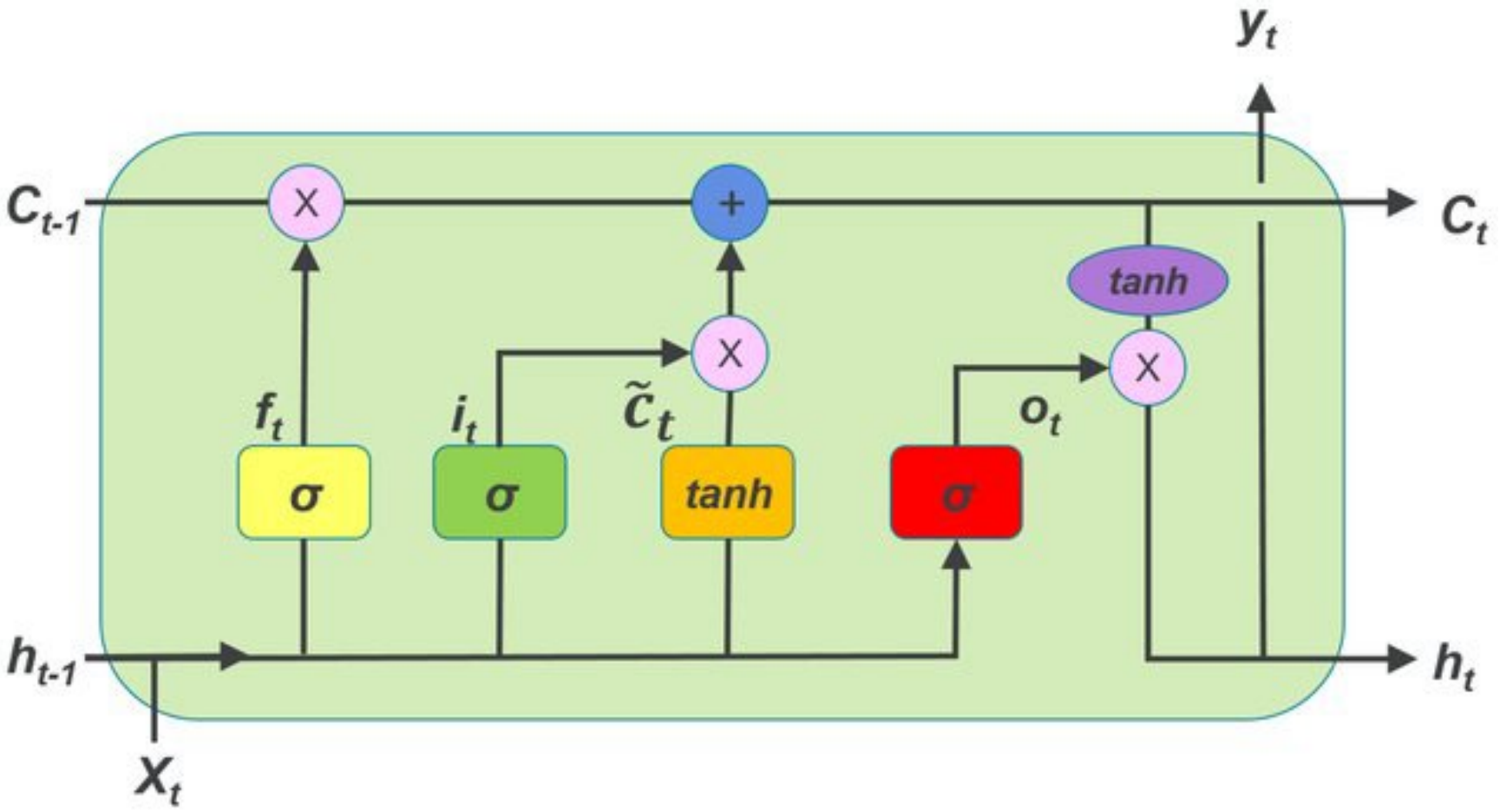
The forward pass of an LSTM cell with a forget gate are

$$
\begin{aligned}
f_t &= \sigma_g(W_f x_t + U_f h_{t-1} + b_f) \\
i_t &= \sigma_g(W_i x_t + U_i h_{t-1} + b_i) \\
o_t &= \sigma_g(W_o x_t + U_o h_{t-1} + b_o) \\
\tilde{c}_t &= \sigma_c(W_c x_t + U_c h_{t-1} + b_c) \\
c_t &= f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \\
h_t &= o_t \odot \sigma_h(c_t)
\end{aligned}
$$

where the initial values are $\boldsymbol{c_0} = \mathbf{0}$ and $\boldsymbol{h_0} = \mathbf{0}$, and the operator $\odot$ denotes the Hadamard product (element-wise product).

- $x_t \in \mathbb{R}^d$: input vector to the LSTM unit
- $f_t \in (0,1)^h$: forget gate's activation vector
- $i_t \in (0,1)^h$: input/update gate's activation vector
- $o_t \in (0,1)^h$: output gate's activation vector
- $h_t \in (-1,1)^h$: hidden state vector
- $\tilde{c}_t \in (-1,1)^h$: cell input activation vector
- $c_t \in \mathbb{R}^h$: cell state vector

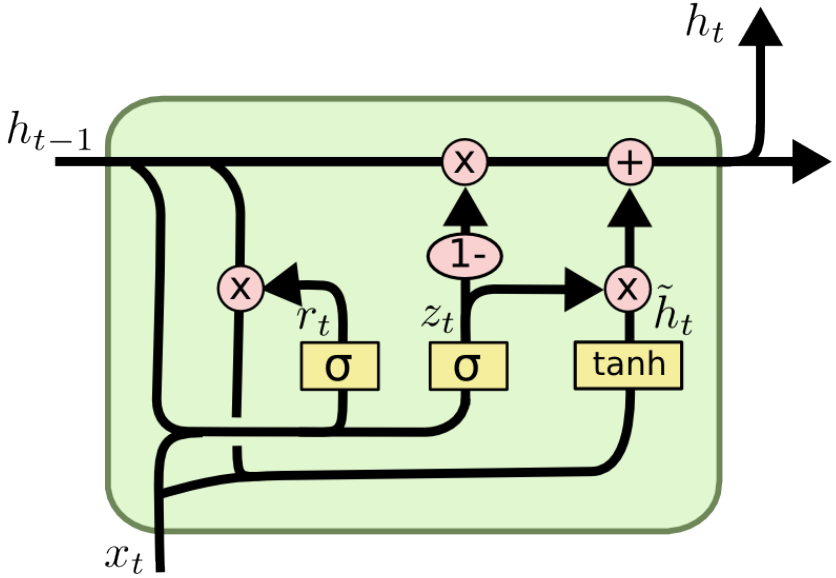
Long Short-Term Memory



Source: https://www.researchgate.net/publication/324600237_Improving_Long-Horizon_Forecasts_with_Expectation-Biased_LSTM_Networks

Gated Recurrent Unit

Wikipedia: Gated Recurrent Unit



$$z_t = \sigma\left(W_z \cdot [h_{t-1}, x_t]\right)$$

$$r_t = \sigma\left(W_r \cdot [h_{t-1}, x_t]\right)$$

$$\tilde{h}_t = \tanh\left(W \cdot [r_t * h_{t-1}, x_t]\right)$$

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$

Source: https://stackoverflow.com/questions/57448101/discrepancy-between-diagram-and-equations-of-gru

Word Embedding

Word Embedding

- In natural language processing (NLP), a word embedding is a representation of a word.
- The embedding is used in text analysis.
- Typically, the representation is a real-valued vector that encodes the meaning of the word in such a way that words that are closer in the vector space are expected to be similar in meaning.

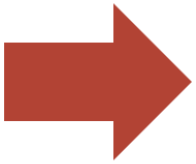
One-hot word embedding

- In a one-hot encoding, or “1-of-N” encoding, the embedding space has the same number of dimensions as the number of words in the vocabulary
- Each word embedding is predominantly made up of zeros, with a “1” in the corresponding dimension for the word.

One-hot word embedding

Example of a one-hot embedding scheme for a nine-word vocabulary. Word embeddings are read as rows of this table.

Vocabulary:
Man, woman, boy, girl, prince, princess, queen, king, monarch



	1	2	3	4	5	6	7	8	9
man	1	0	0	0	0	0	0	0	0
woman	0	1	0	0	0	0	0	0	0
boy	0	0	1	0	0	0	0	0	0
girl	0	0	0	1	0	0	0	0	0
prince	0	0	0	0	1	0	0	0	0
princess	0	0	0	0	0	1	0	0	0
queen	0	0	0	0	0	0	1	0	0
king	0	0	0	0	0	0	0	1	0
monarch	0	0	0	0	0	0	0	0	1

Each word gets a 1x9 vector representation

Source: https://www.shanelynn.ie/get-busy-with-word-embeddings-introduction/

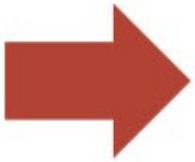
One-hot word embedding

- **The number of dimensions, increases linearly as we add words to the vocabulary.** For a vocabulary of 50,000 words, each word is represented with 49,999 zeros, and a single “one” value in the correct location. The memory use is prohibitively large.
- **The embedding matrix is very sparse, mainly made up of zeros.**
- **There is no shared information between words and no commonalities between similar words.** All words are the same “distance” apart in the embedding space.

Custom embedding

Manually choosing dimensions that make sense for the vocabulary.

Vocabulary:
Man, woman, boy, girl, prince, princess, queen, king, monarch



	Femininity	Youth	Royalty
Man	0	0	0
Woman	1	0	0
Boy	0	1	0
Girl	1	1	0
Prince	0	1	1
Princess	1	1	1
Queen	1	0	1
King	0	0	1
Monarch	0.5	0.5	1

Each word gets a 1x3 vector

Similar words... similar vectors

Source: https://www.shanelynn.ie/get-busy-with-word-embeddings-introduction/

Custom embedding

1. The set of embeddings is more efficient, each word is represented with a 3-dimensional vector.

2. Similar words have similar vectors here. i.e. there's a smaller distance between the embeddings for "girl" and "princess", than from "girl" to "prince". In this case, distance is defined by Euclidean distance.

3. The embedding matrix is much less sparse, and we could potentially add further words to the vocabulary without increasing the dimensionality. For instance, the word "child" might be represented with [0.5, 1, 0].

4. Relationships between words are captured and maintained, e.g. the movement from king to queen, is the same as the movement from boy to girl, and could be represented by [+1, 0, 0].

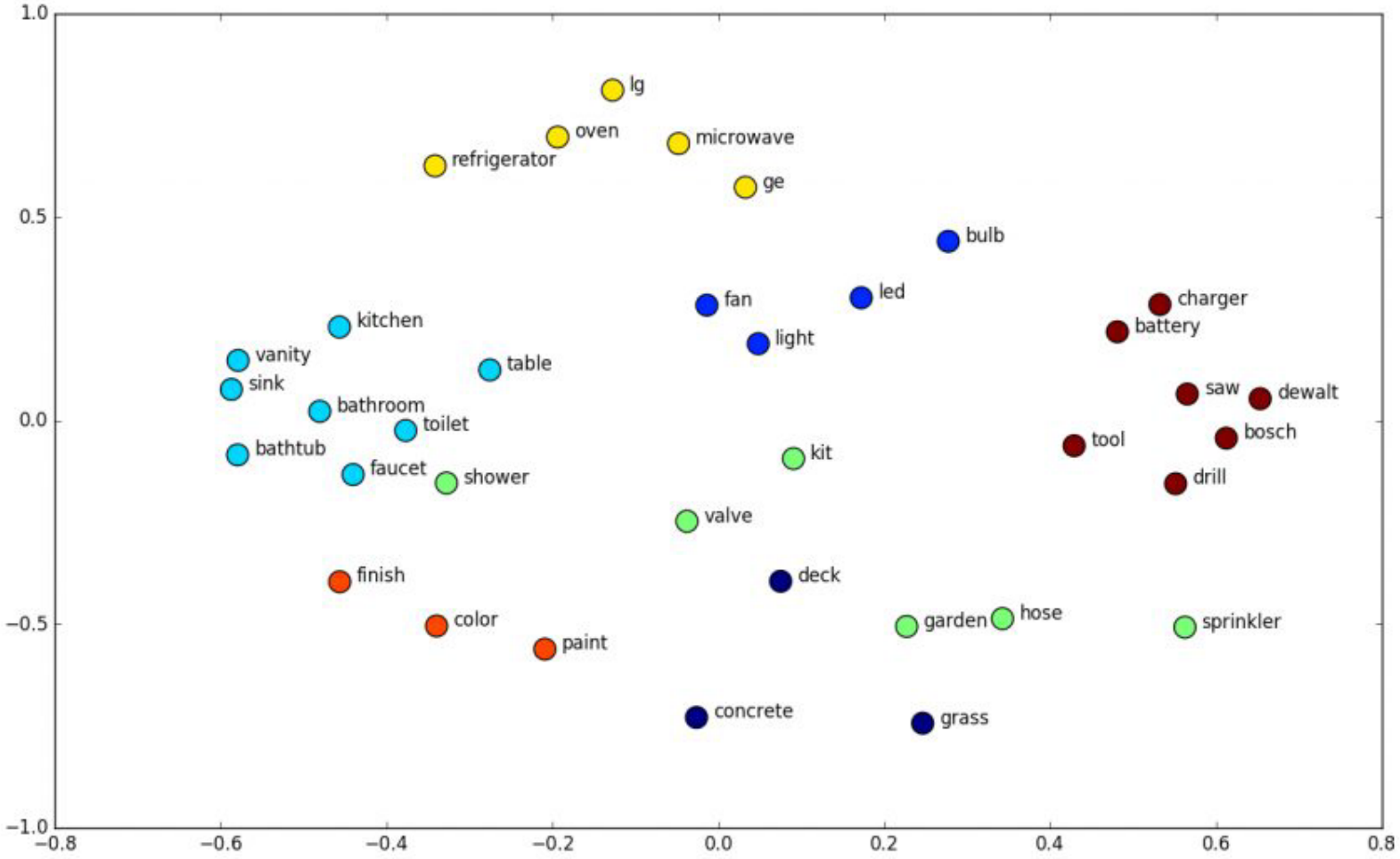
Extending to larger vocabularies

Word Embeddings Properties

- Word Similarities / Synonyms
- Linguistic Relationships

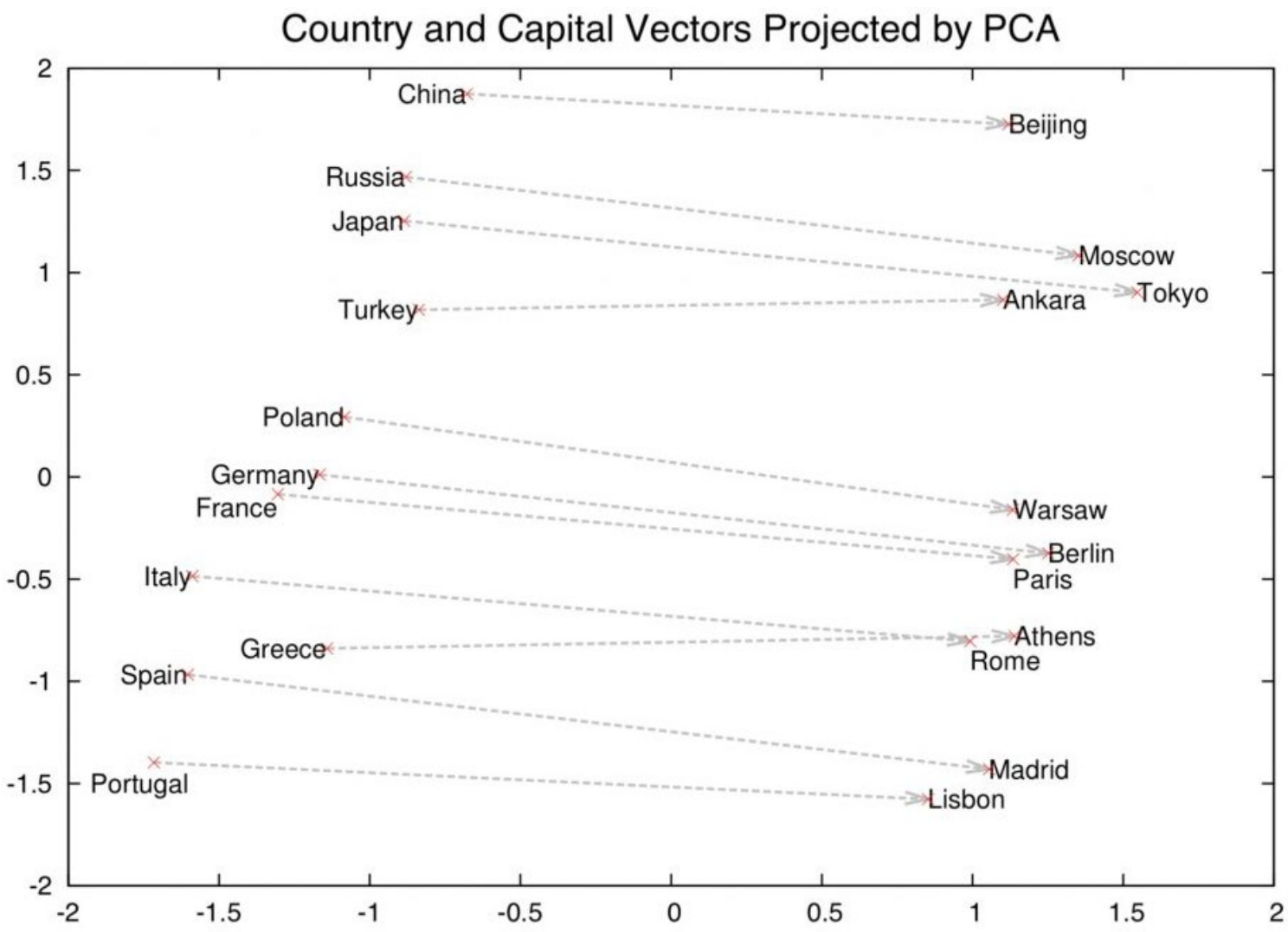
Word Similarities

Example 2D word embedding space, where similar words are found in similar locations.



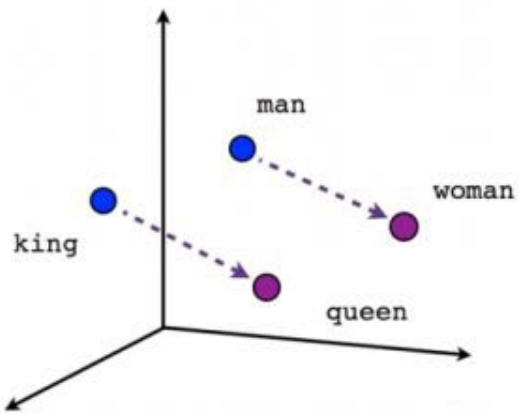
Source: http://suriyadeepan.github.io

Linguistic Relationships

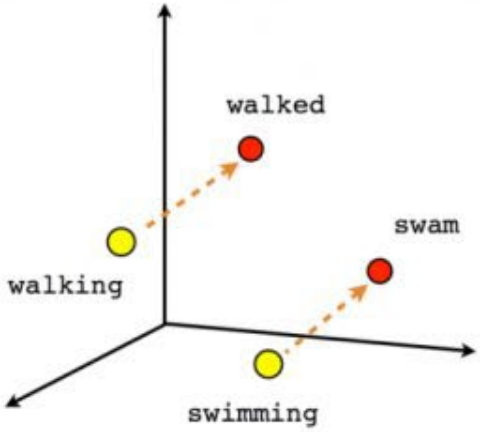


2D PCA projection of word embeddings showing the linear “capital city of” relationship captured by the word-embedding training process.

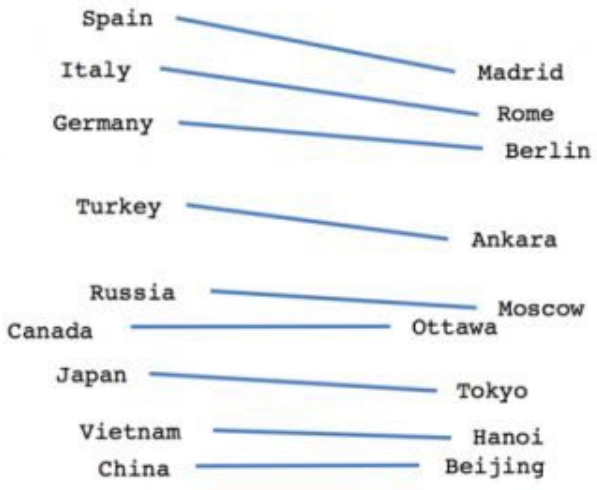
Linguistic Relationships



Male-Female



Verb tense



Country-Capital

Three examples of relationships that are automatically uncovered during word-embedding training – male-female, verb tense, and country-capital.

Extending to larger vocabularies

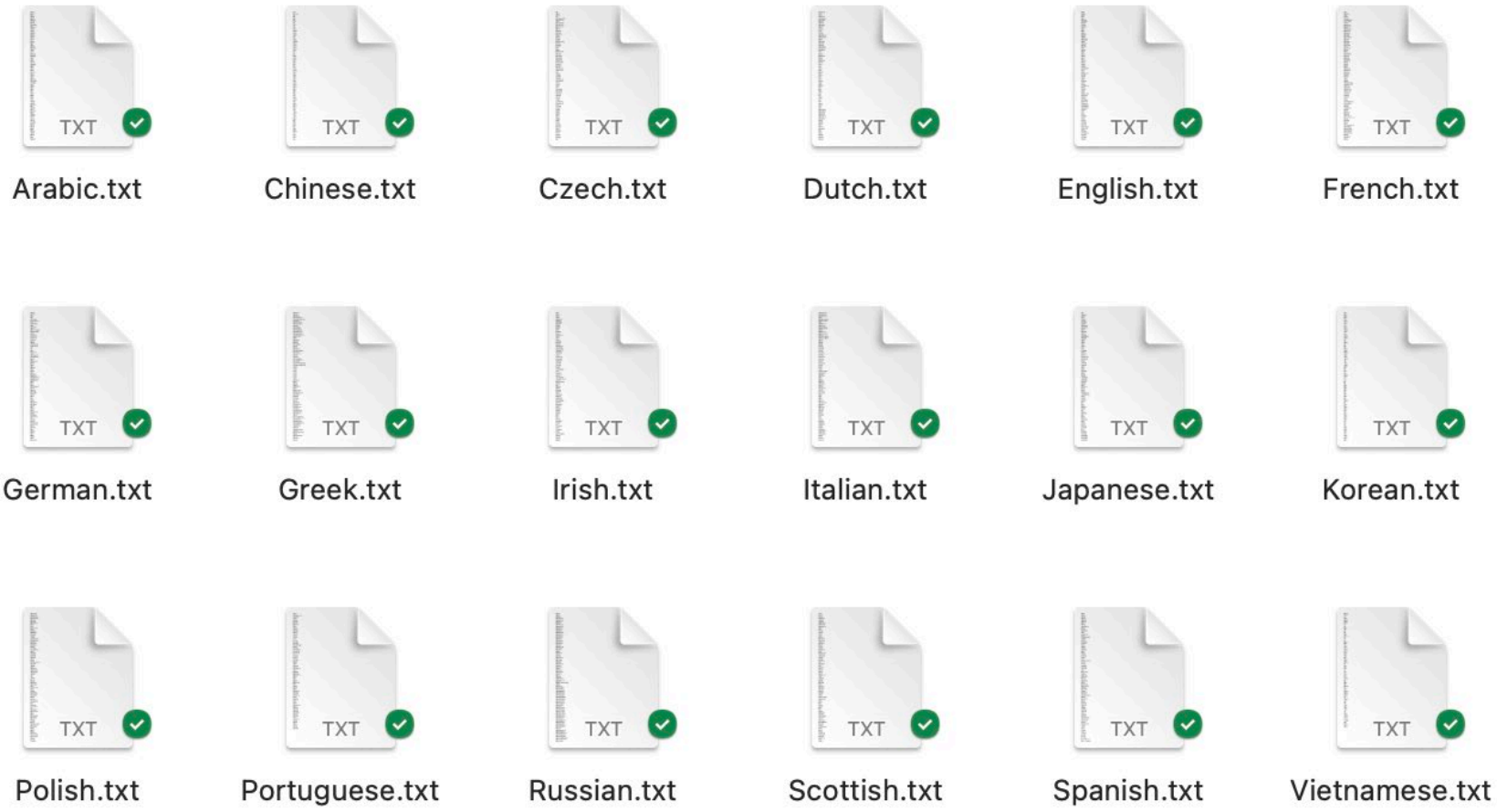
The most popular algorithms include

1. Word2Vec algorithm from Google
2. GloVe algorithm from Stanford
3. fasttext algorithm from Facebook

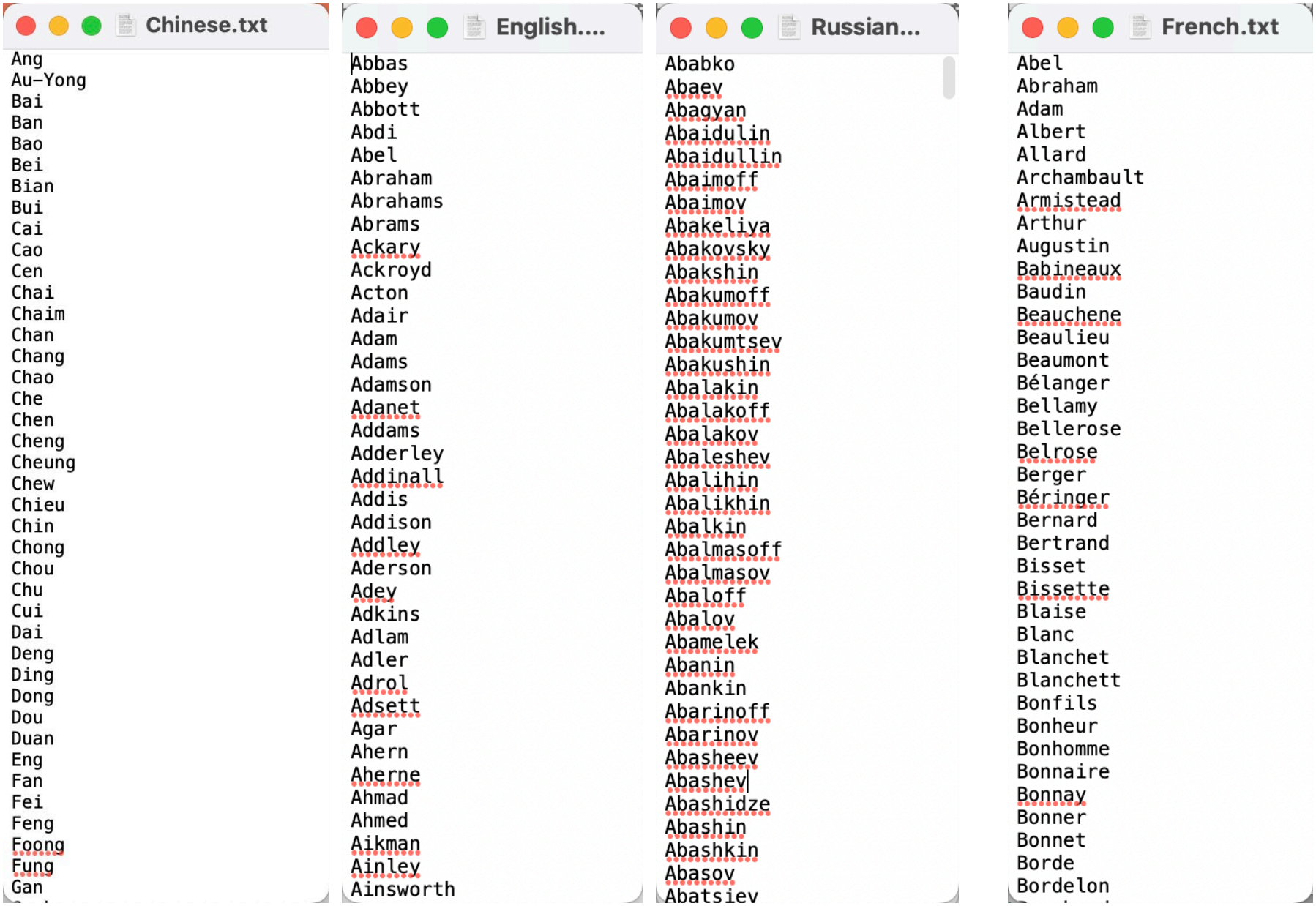
Example

Example: Classification with RNN

Data set: surnames from 18 countries



Example: Classification with RNN



Chinese.txt

Ang
Au-Yong
Bai
Ban
Bao
Bei
Bian
Bui
Cai
Cao
Cen
Chai
Chaim
Chan
Chang
Chao
Che
Chen
Cheng
Cheung
Chew
Chieu
Chin
Chong
Chou
Chu
Cui
Dai
Deng
Ding
Dong
Dou
Duan
Eng
Fan
Fei
Feng
Foong
Fung
Gan

English....

Abbas
Abbey
Abbott
Abdi
Abel
Abraham
Abrahams
Abrams
Ackary
Ackroyd
Acton
Adair
Adam
Adams
Adamson
Adanet
Addams
Adderley
Addinall
Addis
Addison
Addley
Aderson
Adey
Adkins
Adlam
Adler
Adrol
Adsett
Agar
Ahern
Aherne
Ahmad
Ahmed
Aikman
Ainley
Ainsworth

Russian...

Ababko
Abaev
Abagyan
Abaidulin
Abaidullin
Abaimoff
Abaimov
Abakeliya
Abakovsky
Abakshin
Abakumoff
Abakumov
Abakumtsev
Abakushin
Abalakin
Abalakoff
Abalakov
Abaleshev
Abalihin
Abalikhin
Abalkin
Abalmasoff
Abalmasov
Abaloff
Abalov
Abamelek
Abanin
Abankin
Abarinoff
Abarinov
Abasheev
Abashev
Abashidze
Abashin
Abashkin
Abasov
Abatsiev

French.txt

Abel
Abraham
Adam
Albert
Allard
Archambault
Armistead
Arthur
Augustin
Babineaux
Baudin
Beauchene
Beaulieu
Beaumont
Bélanger
Bellamy
Bellerose
Belrose
Berger
Béringer
Bernard
Bertrand
Bisset
Bissette
Blaise
Blanc
Blanchet
Blanchett
Bonfils
Bonheur
Bonhomme
Bonnaire
Bonnay
Bonner
Bonnet
Borde
Bordelon

Example: Classification with RNN

Target:

Train a classifier using RNN to classify surname into 18 country categories

Example: Classification with RNN

Embedding: One-hot embedding

```
import unicodedata
import string

all_letters = string.ascii_letters + " .,;'"
n_letters = len(all_letters)

print(all_letters)
print(n_letters)
```

```
abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ .,;'
57
```

Example: Classification with RNN

Embedding: One-hot embedding

```
# Find letter index from all_letters, e.g. "a" = 0
def letterToIndex(letter):
    return all_letters.find(letter)

print(all_letters.find('a'))
print(all_letters.find('A'))
print(all_letters.find('J'))
```

```
0
26
35
```

Example: Classification with RNN

Embedding: One-hot embedding

```
import torch
# Just for demonstration, turn a letter into a <1 x n_letters> Tensor
def letterToTensor(letter):
    tensor = torch.zeros(1, n_letters)
    tensor[0][letterToIndex(letter)] = 1
    return tensor

print(letterToTensor('a'))
print(letterToTensor('B'))
print(letterToTensor('J'))
```

```
tensor([[1., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.,
         0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.,
         0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.,
         0., 0., 0.]])
tensor([[0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.,
         0., 0., 0., 0., 0., 0., 0., 0., 1., 0., 0., 0., 0., 0., 0., 0., 0.,
         0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.,
         0., 0., 0.]])
tensor([[0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.,
         0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 1.,
         0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.,
         0., 0., 0.]])
```

Example: Classification with RNN

Build RNN

```
class RNN(nn.Module):
    def __init__(self, input_size, hidden_size, output_size):
        super(RNN, self).__init__()
        self.hidden_size = hidden_size
        self.i2h = nn.Linear(input_size + hidden_size, hidden_size)
        self.i2o = nn.Linear(input_size + hidden_size, output_size)
        self.softmax = nn.LogSoftmax(dim=1)

    def forward(self, input, hidden):
        combined = torch.cat((input, hidden), 1)
        hidden = self.i2h(combined)
        output = self.i2o(combined)
        output = self.softmax(output)
        return output, hidden

    def initHidden(self):
        return torch.zeros(1, self.hidden_size)

n_hidden = 128
rnn = RNN(n_letters, n_hidden, n_categories)
```

Example: Classification with RNN

Forward pass in RNN

```
input = letterToTensor('A')
hidden = torch.zeros(1, n_hidden)

output, next_hidden = rnn(input, hidden)
print(output)
```

```
tensor([[-3.1146, -3.9119, -2.8865, -2.4666, -2.9795, -3.5882, -3.4682, -3.0312,
         -3.1896, -2.5168, -3.0321, -3.2578, -2.9676, -2.1635, -2.5508, -2.7902,
         -2.6268, -2.9879]], grad_fn=<LogSoftmaxBackward0>)
```

Example: Classification with RNN

Train the model

```
criterion = nn.NLLLoss(). # Negative log likelihood loss
```

```
learning_rate = 0.005 # If you set this too high, it might explode.
                      # If too low, it might not learn

def train(category_tensor, line_tensor):
    hidden = rnn.initHidden()
    rnn.zero_grad()

    for i in range(line_tensor.size()[0]):
        output, hidden = rnn(line_tensor[i], hidden)

    loss = criterion(output, category_tensor)
    loss.backward()

    # Add parameters' gradients to their values, multiplied by learning rate
    for p in rnn.parameters():
        p.data.add_(p.grad.data, alpha=-learning_rate)

    return output, loss.item()
```

Example: Classification with RNN

Training Process

```
5000 2% (0m 6s) 0.0121 Brisimitzakis / Greek ✓
10000 5% (0m 12s) 2.3322 Wizner / German ✗ (Czech)
15000 7% (0m 19s) 1.5737 Alesio / Spanish ✗ (Italian)
20000 10% (0m 25s) 2.5522 Horner / German ✗ (English)
25000 12% (0m 32s) 0.2332 Giang / Vietnamese ✓
30000 15% (0m 38s) 2.0818 Jorda / Polish ✗ (Spanish)
35000 17% (0m 45s) 0.3461 Ly / Vietnamese ✓
40000 20% (0m 51s) 1.4224 Lindsay / English ✗ (Scottish)
45000 22% (0m 59s) 0.7342 Daher / Arabic ✓
50000 25% (1m 5s) 1.5104 Bauer / Arabic ✗ (German)
55000 27% (1m 12s) 0.4402 Paquet / French ✓
60000 30% (1m 18s) 1.0985 Kuang / Chinese ✓
65000 32% (1m 25s) 1.9924 Caiazzo / Portuguese ✗ (Italian)
70000 35% (1m 31s) 0.0066 Maolmhuaidh / Irish ✓
75000 37% (1m 38s) 0.5535 Dao / Vietnamese ✓
80000 40% (1m 44s) 0.0000 Cuidightheach / Irish ✓
85000 42% (1m 51s) 0.4375 Park  / Korean ✓
90000 45% (1m 57s) 0.2562 Thai / Vietnamese ✓
95000 47% (2m 3s) 0.0537 Zielinski / Polish ✓
100000 50% (2m 10s) 0.0243 Hlutkov / Russian ✓
```

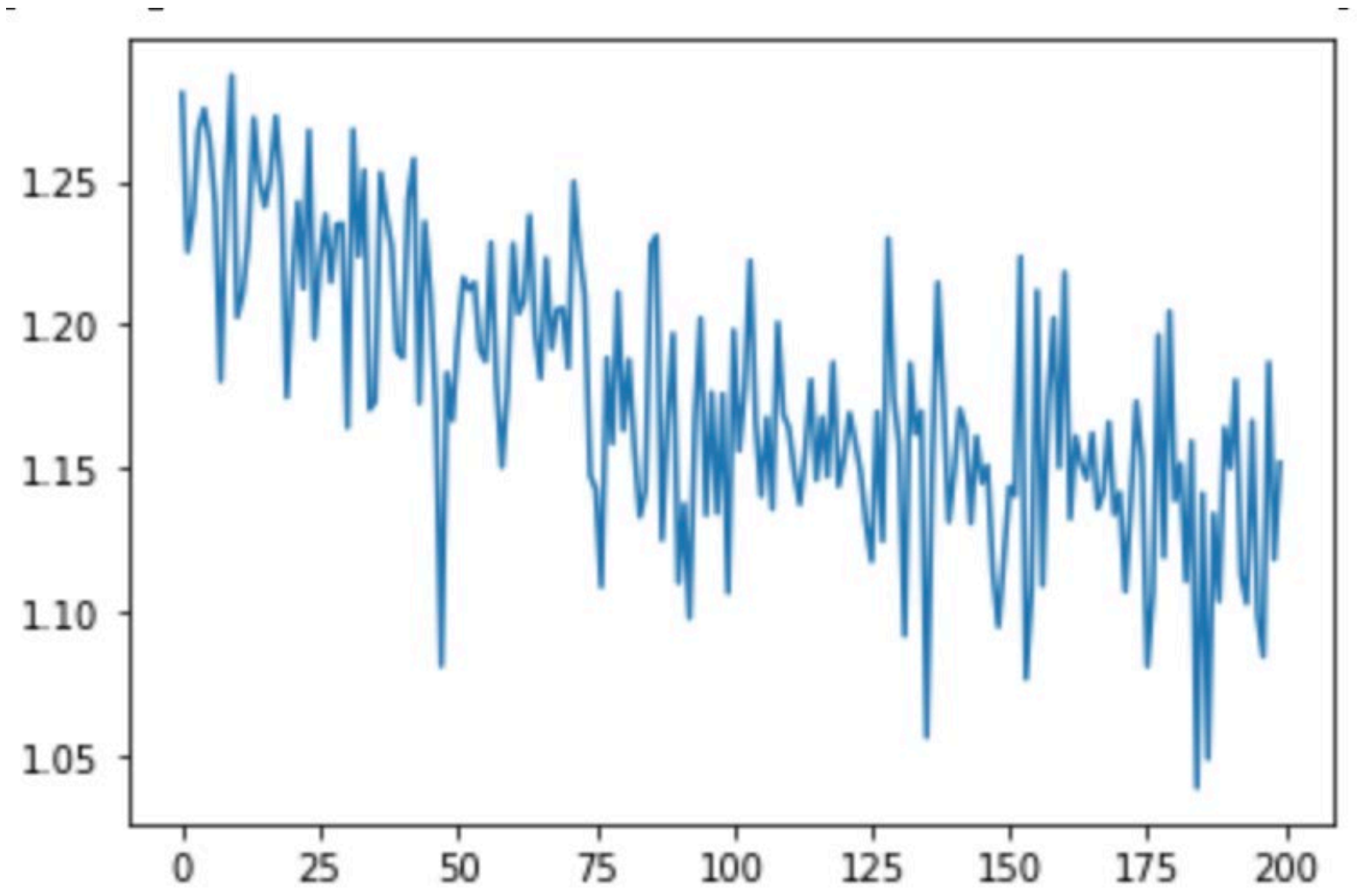
Example: Classification with RNN

Training Process

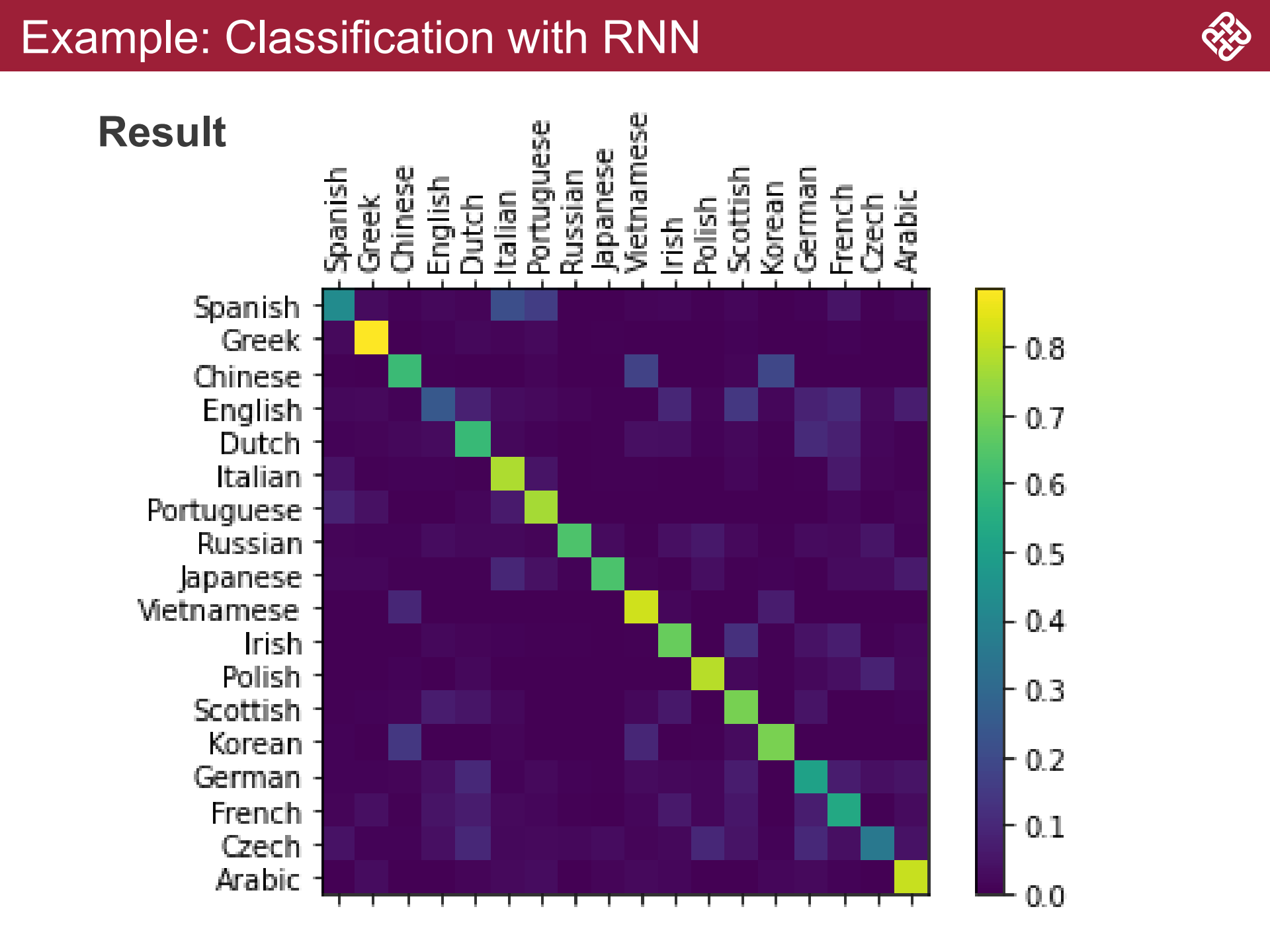
```
105000 52% (2m 16s) 1.2543 Shu / Korean ✗ (Chinese)
110000 55% (2m 23s) 0.3678 Thean / Chinese ✓
115000 57% (2m 28s) 0.8697 Medeiros / Portuguese ✓
120000 60% (2m 35s) 2.3523 Miazga / Japanese ✗ (Polish)
125000 62% (2m 41s) 0.1248 Malinowski / Polish ✓
130000 65% (2m 48s) 0.3051 Santana / Portuguese ✓
135000 67% (2m 54s) 4.7025 Samuel / Arabic ✗ (Irish)
140000 70% (3m 1s) 0.1119 Morrison / Scottish ✓
145000 72% (3m 7s) 0.7417 Cremonesi / Italian ✓
150000 75% (3m 14s) 1.6909 Moreno / Spanish ✗ (Portuguese)
155000 77% (3m 20s) 1.5550 Jez / Chinese ✗ (Polish)
160000 80% (3m 27s) 4.0158 Chromy / Irish ✗ (Czech)
165000 82% (3m 33s) 0.2522 O'Donnell / Irish ✓
170000 85% (3m 40s) 0.3419 Airaldi / Italian ✓
175000 87% (3m 46s) 1.5858 Robles / Spanish ✓
180000 90% (3m 53s) 0.0202 Arnoni / Italian ✓
185000 92% (3m 59s) 2.8040 Legg / Vietnamese ✗ (English)
190000 95% (4m 6s) 1.7104 Oliver / French ✗ (Spanish)
195000 97% (4m 12s) 0.4689 Luc / Vietnamese ✓
200000 100% (4m 18s) 5.7895 Tsoumada / Japanese ✗ (Greek)
```

Example: Classification with RNN

Training Loss



Example: Classification with RNN



Result

Spanish
Greek
Chinese
English
Dutch
Italian
Portuguese
Russian
Japanese
Vietnamese
Irish
Polish
Scottish
Korean
German
French
Czech
Arabic

0.8
0.7
0.6
0.5
0.4
0.3
0.2
0.1
0.0

Example: Classification with RNN

Result

```
> Dovesky
(-0.87) Russian
(-0.96) Czech
(-1.84) English

> Jackson
(-1.35) English
(-1.59) Scottish
(-1.73) Czech

> Satoshi
(-0.83) Arabic
(-1.31) Italian
(-1.74) Japanese
```

```
> Wang
(-0.58) Chinese
(-1.42) Korean
(-2.42) Scottish

> Peterson
(-0.63) Scottish
(-1.75) Dutch
(-1.85) German

> Abraham
(-1.47) Russian
(-1.64) English
(-2.00) Vietnamese
```